



PROJET PROPEL

Rapport fonctionnel

REALISER PAR

Bilal Magguilej
Nourelhouda Ouhba
Mouhssine El Haouary
Mohssine El Addaoui
Salman El-Ghrich
Somia Jdia

ENCADRER PAR

M.GHAILANI

 +212 602-115105

 <https://github.com/BilalMagguilej/CV-Generator>

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réussite de ce projet d'intégration.

Nous remercions chaleureusement **M. GHAILANI**, professeur responsable du module, pour son encadrement, ses orientations précieuses et son suivi rigoureux tout au long du semestre. Ses conseils nous ont permis de maintenir le cap et de structurer notre travail de manière professionnelle.

Nous adressons également nos sincères remerciements aux **élèves ingénieurs du niveau supérieur** ainsi qu'aux **lauréats** qui ont partagé leurs retours d'expérience et leurs recommandations, contribuant ainsi à l'amélioration continue de notre projet.

Nous tenons à exprimer notre reconnaissance aux **membres du jury** pour l'attention qu'ils porteront à l'évaluation de notre travail et pour le temps qu'ils nous accorderont lors de la soutenance.

Enfin, nous remercions tous ceux qui, de près ou de loin, ont contribué à la réalisation de ce projet par leur soutien, leurs encouragements ou leur expertise.

L'équipe CV_Generator

Résumé

Ce projet présente la conception et le développement de **Propel**, une plateforme web intelligente de génération de CV assistée par intelligence artificielle. L'application permet aux utilisateurs de générer un curriculum vitae personnalisé et optimisé pour une offre d'emploi spécifique en quelques secondes. Le système repose sur une architecture en microservices combinant Angular pour le frontend, ASP.NET Core pour les services backend, et une pipeline d'agents IA en Python utilisant LangChain et LangGraph pour l'extraction, l'analyse, l'optimisation et la livraison des CV. L'infrastructure est entièrement conteneurisée avec Docker et déployée via un pipeline CI/CD complet intégrant des outils de sécurité et de qualité (SonarCloud, Trivy, OWASP ZAP). Le projet a été réalisé par une équipe de six développeurs fullstack selon une méthodologie SCRUM adaptée.

Mots-clés : Génération de CV, Intelligence Artificielle, Microservices, Angular, ASP.NET Core, LangChain, Docker, DevSecOps.

Abstract

This project presents the design and development of **Propel**, an intelligent web platform for AI-powered CV generation. The application enables users to generate a tailored and optimized curriculum vitae for a specific job offer in seconds. The system is built on a microservices architecture combining Angular for the frontend, ASP.NET Core for backend services, and a Python AI agent pipeline using LangChain and LangGraph for extraction, analysis, optimization, and delivery of CVs. The infrastructure is fully containerized with Docker and deployed through a complete CI/CD pipeline integrating security and quality tools (SonarCloud, Trivy, OWASP ZAP). The project was carried out by a team of six full-stack developers following an adapted SCRUM methodology.

Keywords : CV Generation, Artificial Intelligence, Microservices, Angular, ASP.NET Core, LangChain, Docker, DevSecOps.

Liste des acronymes

.NET Microsoft .NET Framework.

Angular Angular Framework.

API Application Programming Interface.

ATS Applicant Tracking System.

CD Continuous Deployment.

CI Continuous Integration.

CRUD Create, Read, Update, Delete.

DAST Dynamic Application Security Testing.

DNS Domain Name System.

E2E End-to-End.

gRPC gRPC Remote Procedure Call.

HTTP HyperText Transfer Protocol.

HTTPS HyperText Transfer Protocol Secure.

IA Intelligence Artificielle.

IaC Infrastructure as Code.

JSON JavaScript Object Notation.

JWT JSON Web Token.

Kafka Apache Kafka.

LLM Large Language Model.

OAuth Open Authorization.

OIDC OpenID Connect.

OWASP Open Web Application Security Project.

PDF Portable Document Format.

PR Pull Request.

RAG Retrieval-Augmented Generation.

REST Representational State Transfer.

S3 Simple Storage Service.

SOLID Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion.

SPA Single Page Application.

SQL Structured Query Language.

SSO Single Sign-On.

VM Virtual Machine.

XSS Cross-Site Scripting.

YARP Yet Another Reverse Proxy.

Table des matières

Remerciements	1
Résumé	2
Abstract	3
Liste des acronymes	4
1 Introduction générale	11
2 Contexte général du projet	12
2.1 Introduction	12
2.2 Présentation du projet	12
2.2.1 Cadre du projet	12
2.2.2 Objectifs du projet	12
2.2.3 Méthodologie de gestion de projet	13
2.2.4 Présentation de l'équipe	13
2.2.5 Planification du projet	15
2.3 Conclusion	15
3 Analyse et conception	16
3.1 Introduction	16
3.2 Étude de l'existant	16
3.3 Étude des besoins fonctionnels	17
3.4 Étude des besoins non fonctionnels	18
3.5 Identification des acteurs	18
3.6 Modélisation du système	19
3.6.1 Diagramme de cas d'utilisation	19
3.6.2 Diagramme de classes	23
3.6.3 Diagramme de séquence — Flux d'authentification	24
3.6.4 Modèle relationnel de la base de données	25
3.7 Conclusion	32

4	Réalisation	33
4.1	Introduction	33
4.2	Étude technique	33
4.2.1	Architecture physique	33
4.2.2	Architecture logicielle	37
4.2.3	Présentation des outils utilisés	39
4.3	Cycle DevSecOps	40
4.3.1	Présentation et importance dans le projet	40
4.3.2	Outils DevSecOps utilisés	40
4.3.3	Application du cycle DevSecOps dans le projet	40
4.3.4	Sécurité applicative	41
4.4	Gestion du code source	41
4.4.1	Utilisation de Git	41
4.4.2	Organisation des branches	41
4.4.3	Gestion des commits et collaboration	41
4.5	Implémentation	42
4.5.1	Développement Backend	42
4.5.2	Pipeline de génération de CV	43
4.5.3	Développement Frontend	44
4.5.4	Base de données	45
4.6	Déploiement	45
4.6.1	Mise en place de l'environnement de développement	45
4.6.2	Dockerisation de l'application	45
4.6.3	Pipeline CI/CD	45
4.7	Présentation de l'application	46
4.7.1	Interface utilisateur	46
4.7.2	Fonctionnalités principales	46
4.7.3	Démonstrations du fonctionnement	46
4.8	Assurance Qualité	46
4.8.1	Tests Unitaires	47
4.8.2	Tests d'Intégration	49
4.8.3	Tests End-to-End (E2E) — Cypress	50
4.8.4	Tests de Performance — k6	52
4.9	Conclusion	56
5	Conclusion générale et perspectives	57
5.1	Conclusion générale	57
5.2	Perspectives	57

Table des figures

2.1	Organigramme fonctionnel de l'équipe	15
2.2	Diagramme de Gantt prévisionnel	15
2.3	Diagramme de Gantt réel	15
3.1	Cas d'utilisation — Authentification	19
3.2	Cas d'utilisation — Gestion du profil	20
3.3	Cas d'utilisation — Génération de CV	21
3.4	Cas d'utilisation — Suivi des candidatures	22
3.5	Cas d'utilisation — Offres d'emploi et notifications	23
3.6	Diagramme de classes — Relations inter-services	23
3.7	Diagramme de séquence — Flux d'authentification OIDC	24
3.8	Modèle relationnel — Base Utilisateurs	26
3.9	Modèle relationnel — Base Contenu Utilisateur	27
3.10	Modèle relationnel — Base Candidatures	28
3.11	Modèle relationnel — Base Workflow	29
3.12	Modèle relationnel — Base Offres d'Emploi	30
3.13	Modèle relationnel — Base CV	31
3.14	Modèle relationnel — Base Notifications	32
4.1	Architecture de déploiement locale — Répartition des VMs par membre	34
4.2	Architecture logicielle du système	38
4.3	Organisation des branches Git	41
4.4	Flux des événements Kafka — Publications et consommations	43
4.5	Pipeline de génération de CV — De l'offre d'emploi à l'envoi par email	44
4.6	Pipeline CI/CD	45
4.7	Interface du tableau de bord	46
4.8	Interface de génération de CV	46
4.9	Tableau Kanban de suivi des candidatures	46
4.10	Interface de gestion du portfolio	46
4.11	Résultats de l'exécution des tests pytest — contact-agent	48
4.12	Résultats de l'exécution des tests xUnit — ApplicationService	48

4.13 Résultats des tests d'intégration — Testcontainers + FastAPI TestClient	49
4.14 Interface Cypress — Exécution de la suite E2E avec 24 tests passants .	51
4.15 Détail du fichier <code>auth.cy.ts</code> — Scénarios d'authentification	52
4.16 Sortie k6 — Scénario <code>register.js</code> : création d'utilisateurs sous charge (10 VUs)	54
4.17 Sortie k6 — Scénario <code>login.js</code> avec métriques et seuils	55
4.18 Sortie k6 — Scénario <code>me.js</code> : validation du cookie de session (p95 < 500ms)	56

Liste des tableaux

2.1	Répartition des rôles au sein de l'équipe	14
3.1	Comparaison des solutions existantes	16
3.2	Besoins fonctionnels du système	17
3.3	Identification des acteurs du système	18
4.1	Services AWS utilisés dans l'infrastructure cloud	35
4.2	Outils de développement	39
4.3	Outils CI/CD et qualité	39
4.4	Fichiers de spécifications Cypress et couverture	51
4.5	Scénarios k6 et seuils de performance	53

Chapitre 1

Introduction générale

Dans un monde professionnel de plus en plus compétitif, la qualité et la pertinence d'un curriculum vitae (CV) jouent un rôle déterminant dans le succès d'une candidature. Chaque offre d'emploi possède ses propres exigences, et un CV générique a souvent moins d'impact qu'un CV adapté spécifiquement au poste visé. Cependant, personnaliser manuellement son CV pour chaque candidature est un processus chronophage et fastidieux.

C'est dans ce contexte que s'inscrit le projet **Propel**, une plateforme web intelligente de génération de CV assistée par intelligence artificielle. L'objectif est de permettre aux candidats de générer un CV personnalisé et optimisé pour une offre d'emploi donnée en quelques secondes, tout en assurant une compatibilité optimale avec les systèmes de suivi des candidatures (ATS).

Ce rapport présente l'ensemble du travail réalisé par notre équipe dans le cadre du projet d'intégration de la 1ère année Génie Informatique, sous la supervision de M. GHAILANI.

Le rapport est structuré en trois chapitres principaux. Le premier chapitre pose le contexte général du projet, présente les objectifs, la méthodologie de gestion adoptée ainsi que l'équipe de réalisation. Le deuxième chapitre est consacré à l'analyse et la conception du système, incluant l'étude des besoins, l'identification des acteurs et la modélisation. Enfin, le troisième chapitre détaille la réalisation technique du projet, couvrant l'architecture, les outils utilisés, le cycle DevSecOps, l'implémentation et le déploiement.

Chapitre 2

Contexte général du projet

2.1 Introduction

Ce premier chapitre présente le cadre général du projet **Propel**. Nous y décrivons le contexte dans lequel s’inscrit le projet, les objectifs visés, la méthodologie de gestion adoptée, ainsi que la composition et l’organisation de l’équipe de réalisation.

2.2 Présentation du projet

2.2.1 Cadre du projet

Ce projet est réalisé dans le cadre du module **Projet d’Intégration** de la 1ère année Génie Informatique, sous la supervision de M. GHAILANI. Il constitue une mise en situation professionnelle où les étudiants sont amenés à concevoir, développer et déployer une application complète en appliquant les compétences acquises durant leur formation.

L’équipe est composée de six étudiants, chacun jouant le rôle de développeur full-stack avec une responsabilité secondaire spécifique (CI/CD, infrastructure cloud, infrastructure locale, sécurité, qualité, monitoring), reflétant ainsi une organisation DevOps complète.

2.2.2 Objectifs du projet

Les objectifs principaux du projet **Propel** sont les suivants :

- **Génération intelligente de CV** : Permettre à un utilisateur de soumettre une offre d’emploi et de recevoir un CV personnalisé et optimisé pour cette offre, généré automatiquement par une pipeline d’agents d’intelligence artificielle.
- **Suivi des candidatures** : Offrir un tableau de bord complet avec vue Kanban, liste et calendrier pour gérer l’ensemble des candidatures.

- **Gestion de portfolio** : Permettre à l'utilisateur de gérer ses projets, compétences, expériences et formations.
- **Agrégation d'offres d'emploi** : Collecter automatiquement des offres d'emploi depuis différentes plateformes via un agent de crawling.
- **Notifications temps réel** : Notifier l'utilisateur sur l'état d'avancement de ses candidatures et de ses générations de CV.
- **Déploiement industrialisé** : Mettre en place un pipeline CI/CD complet avec conteneurisation, tests automatisés, analyse de qualité et scans de sécurité.

2.2.3 Méthodologie de gestion de projet

Pour la gestion de ce projet, nous avons adopté une approche **SCRUM** adaptée à notre contexte. Le professeur joue le rôle de **Product Owner**, définissant les objectifs et validant les livrables. Le chef d'équipe assume le rôle de **SCRUM Master**, facilitant la coordination et s'assurant de l'avancement des tâches.

La méthodologie mise en œuvre se caractérise par :

- **Réunions quotidiennes** : Des sessions de coordination via **Google Meets** sont organisées régulièrement pour faire le point sur l'avancement, discuter des obstacles et planifier les tâches à venir.
- **Communication continue** : Un groupe **WhatsApp** dédié permet une communication réactive pour la résolution rapide des problèmes et le partage d'information.
- **Organisation en sprints** : Le projet est découpé en itérations (sprints) avec des objectifs clairs à chaque étape, permettant une livraison incrémentale des fonctionnalités.
- **Gestion des tâches** : La répartition des tâches est définie collectivement lors des réunions, en fonction des priorités et des compétences de chaque membre.

Cette approche agile nous a permis de nous adapter aux défis techniques rencontrés tout au long du projet et de maintenir un rythme de travail soutenu.

2.2.4 Présentation de l'équipe

L'équipe **CV_Generator** est composée de six membres, chacun occupant un rôle de développeur fullstack avec une responsabilité transverse spécifique. Cette organisation permet de couvrir l'ensemble du cycle de vie du développement logiciel, du code au déploiement en passant par la qualité et la sécurité.

TABLE 2.1 – Répartition des rôles au sein de l'équipe

Rôle	Eff.	Intervenant	Remarque
Chef d'équipe / Responsable CI/CD	1	Bilal Magguilej	Coordination d'équipe, pipeline CI/CD, Dockerisation complète, validation des PR
Développeur Fullstack – Infra Cloud	1	Mouhssine El Haouary	Déploiement cloud, environnement de production
Développeur Fullstack – Infra Locale	1	Somia Jdia	Terraform, provisionnement VMs locales, environnement de développement
Développeur Fullstack – Sécurité	1	Nourelhouda Ouhba	Keycloak SSO, OWASP ZAP, Trivy, authentification OAuth
Développeur Fullstack – QA	1	Mohssine El Addaoui	Tests E2E Cypress, tests de charge k6, assurance qualité
Développeur Fullstack – Monitoring	1	Salman El-Ghrich	Prometheus, Grafana, observabilité et métriques

Chaque membre a participé activement au développement des trois couches de l'application : frontend (Angular), backend (.NET) et agents IA (Python/FastAPI). Les responsabilités secondaires ont été réparties pour couvrir l'ensemble des aspects DevOps et assurer la qualité et la maintenabilité du projet.

FIGURE 2.1 – Organigramme fonctionnel de l'équipe

2.2.5 Planification du projet

Le projet a été planifié sur la durée du semestre avec des jalons clés correspondant aux livrables attendus. Le diagramme de Gantt ci-dessous présente la planification prévisionnelle ainsi que le déroulement réel du projet.

FIGURE 2.2 – Diagramme de Gantt prévisionnel

FIGURE 2.3 – Diagramme de Gantt réel

2.3 Conclusion

Ce premier chapitre a présenté le cadre général du projet **Propel**, les objectifs visés, la méthodologie de gestion adoptée ainsi que l'organisation de l'équipe. Le chapitre suivant sera consacré à l'analyse détaillée des besoins et à la conception du système.

Chapitre 3

Analyse et conception

3.1 Introduction

Ce chapitre présente l'analyse approfondie du système **Propel**. Nous commençons par une étude des solutions existantes, suivie de l'identification des besoins fonctionnels et non fonctionnels. Ensuite, nous identifions les acteurs du système avant de présenter les différents diagrammes de modélisation.

3.2 Étude de l'existant

Plusieurs solutions de création de CV existent sur le marché. Le tableau suivant présente une comparaison des principales solutions :

TABLE 3.1 – Comparaison des solutions existantes

Solution	CV per-sonnalisé	IA	Suivi can-didatures	Open Source
Propel (notre solution)	Oui	Oui	Oui	Oui
Canva	Partiel	Non	Non	Non
Zety	Modèles fixes	Non	Non	Non
Novoresume	Modèles fixes	Non	Non	Non
Rezi	Oui	Oui	Non	Non
Enhancv	Partiel	Oui	Non	Non

Les solutions existantes présentent plusieurs limitations :

- **Personnalisation limitée** : La plupart des outils offrent des modèles fixes sans adaptation réelle au contenu de l'offre d'emploi.

- **Absence de suivi intégré** : Aucune solution ne combine la génération de CV avec le suivi des candidatures dans une plateforme unifiée.
- **Coût** : Les solutions avancées sont payantes (abonnements mensuels).
- **Optimisation ATS limitée** : Peu d'outils optimisent réellement le CV pour passer les filtres des systèmes ATS.

Propel se distingue en offrant une solution complète, gratuite et open source, combinant génération IA de CV, optimisation ATS et suivi de candidatures.

3.3 Étude des besoins fonctionnels

Les besoins fonctionnels identifiés pour le système **Propel** sont les suivants :

TABLE 3.2 – Besoins fonctionnels du système

ID	Description
BF01	L'utilisateur doit pouvoir s'authentifier via Keycloak (email, Google, GitHub)
BF02	L'utilisateur doit pouvoir gérer son portfolio (projets, compétences, expériences, formations)
BF03	L'utilisateur doit pouvoir soumettre une offre d'emploi pour générer un CV personnalisé
BF04	Le système doit extraire les informations clés de l'offre d'emploi via un agent IA
BF05	Le système doit rechercher les compétences pertinentes dans le profil de l'utilisateur via RAG
BF06	Le système doit générer un CV formaté selon un template optimisé ATS
BF07	Le système doit optimiser le contenu du CV pour maximiser la compatibilité ATS
BF08	Le système doit envoyer le CV généré par email à l'utilisateur
BF09	L'utilisateur doit pouvoir suivre ses candidatures (vue Kanban, liste, calendrier)
BF10	L'utilisateur doit pouvoir visualiser des statistiques et des graphiques sur ses candidatures
BF11	Le système doit crawler automatiquement des offres d'emploi depuis des plateformes externes
BF12	L'utilisateur doit recevoir des notifications temps réel sur l'état de ses candidatures
BF13	L'utilisateur doit pouvoir gérer plusieurs versions de CV
BF14	Le système doit exporter les CV au format PDF

3.4 Étude des besoins non fonctionnels

Les besoins non fonctionnels identifiés sont :

- **Performance** : La génération d'un CV ne doit pas excéder quelques secondes. Les temps de réponse de l'API doivent être optimisés.
- **Sécurité** : L'authentification doit être centralisée via Keycloak avec support OAuth2 et OpenID Connect. Les communications doivent être chiffrées (HTTPS). Les vulnérabilités doivent être scannées via Trivy et OWASP ZAP.
- **Disponibilité** : Le système doit être résilient grâce à une architecture en microservices et une orchestration Docker. Les services critiques doivent pouvoir redémarrer automatiquement.
- **Maintenabilité** : Le code source doit être organisé en modules séparés, versionné avec Git, et suivre les principes SOLID. La qualité du code est surveillée via SonarCloud.
- **Évolutivité** : L'architecture microservices permet de faire évoluer chaque service indépendamment. L'utilisation de Kafka permet un couplage faible entre les services.
- **Ergonomie** : L'interface utilisateur doit être intuitive, responsive et suivre une charte graphique cohérente (design system Propel).
- **Portabilité** : L'ensemble de l'application doit être conteneurisée avec Docker pour garantir un déploiement reproductible sur n'importe quel environnement.

3.5 Identification des acteurs

L'analyse du système **Propel** permet d'identifier les acteurs suivants :

TABLE 3.3 – Identification des acteurs du système

Acteur	Description
Utilisateur	Candidat cherchant à générer des CV personnalisés et suivre ses candidatures
Système d'authentification	Keycloak gère l'authentification et l'autorisation via OAuth2/OpenID Connect
Pipeline IA	Ensemble d'agents intelligents qui traitent les offres d'emploi et génèrent les CV
Système de messagerie	Kafka assure la communication asynchrone entre les microservices
Administrateur	Gère la configuration du système et supervise le déploiement

3.6 Modélisation du système

3.6.1 Diagramme de cas d'utilisation

Les diagrammes de cas d'utilisation ci-dessous présentent les interactions entre l'utilisateur et les différentes fonctionnalités du système. Cinq diagrammes couvrent les principaux modules de l'application.

Authentification L'utilisateur peut créer un compte, se connecter et se déconnecter du système.

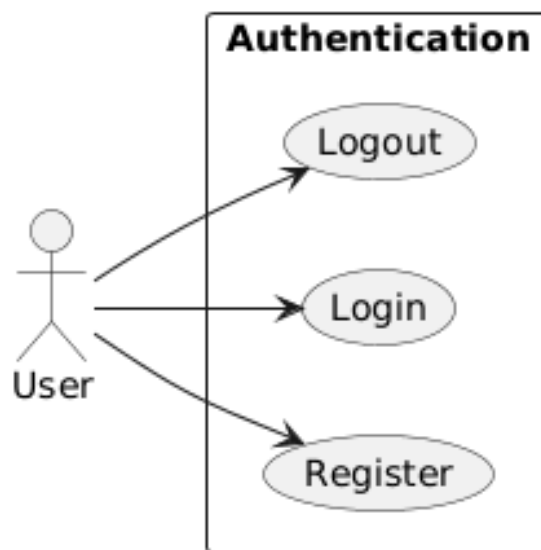


FIGURE 3.1 – Cas d'utilisation — Authentification

Gestion du profil L'utilisateur peut gérer l'ensemble de ses données personnelles et professionnelles : informations de profil, projets, compétences, expériences, formations, certifications, langues, liens sociaux et profils CV. Tous ces cas nécessitent une authentification préalable.

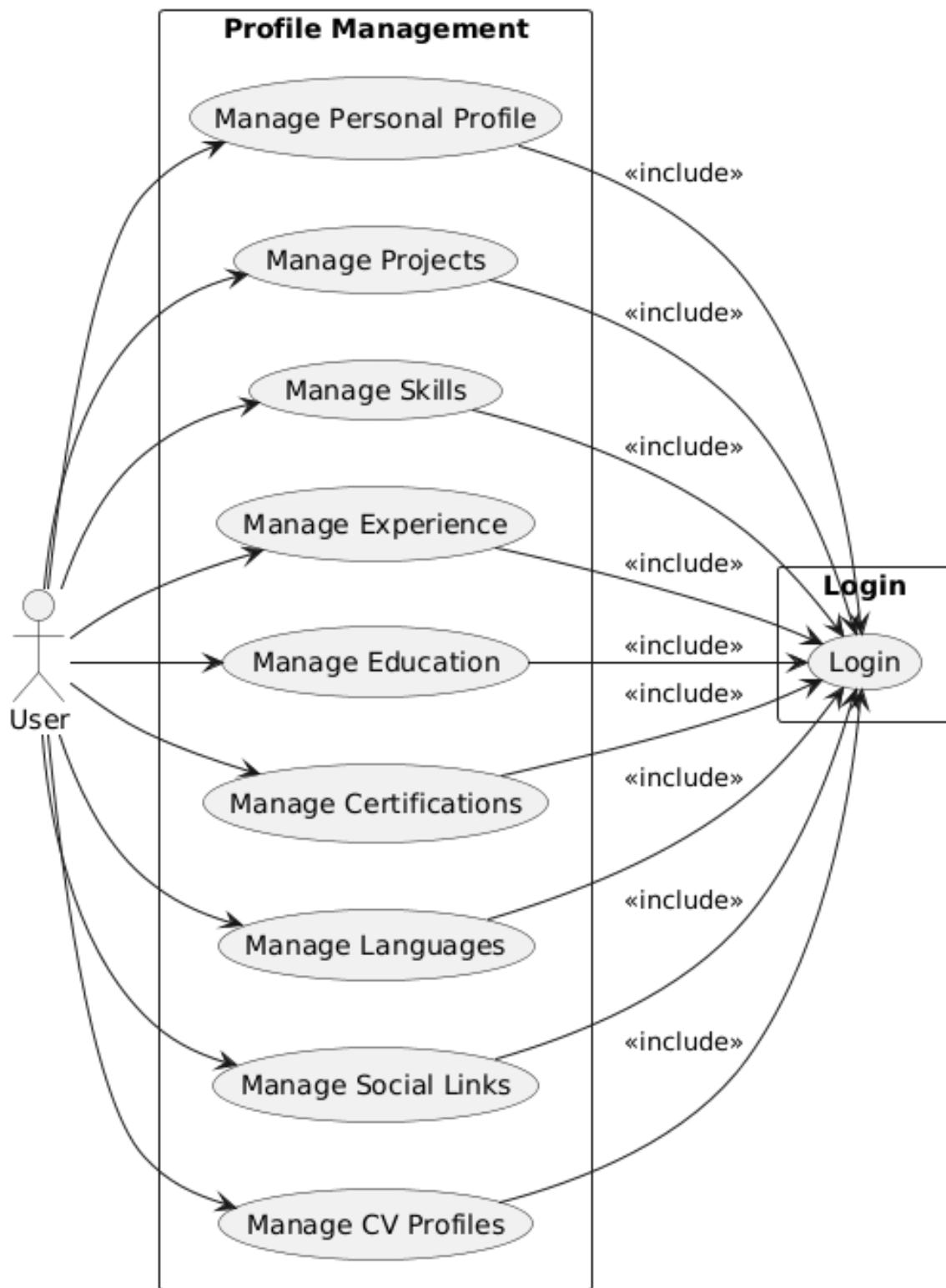


FIGURE 3.2 – Cas d'utilisation — Gestion du profil

Génération de CV L'utilisateur peut générer un CV personnalisé à partir d'une offre d'emploi, choisir un template, définir le ton du contenu, exporter le CV au format PDF et consulter l'historique des générations. Le système d'IA intervient comme acteur

secondaire dans le processus de génération.

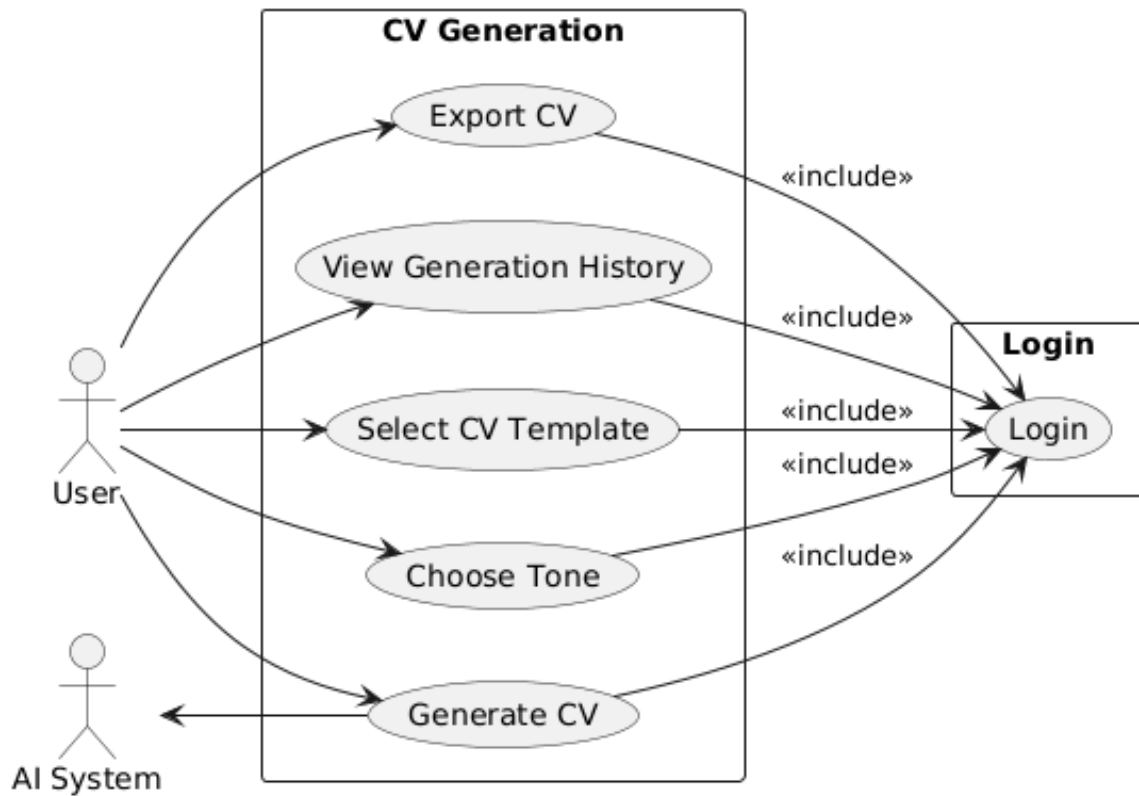


FIGURE 3.3 – Cas d'utilisation — Génération de CV

Suivi des candidatures L'utilisateur peut ajouter une candidature, mettre à jour son statut, visualiser le tableau Kanban, consulter les analyses statistiques et le calendrier des événements.

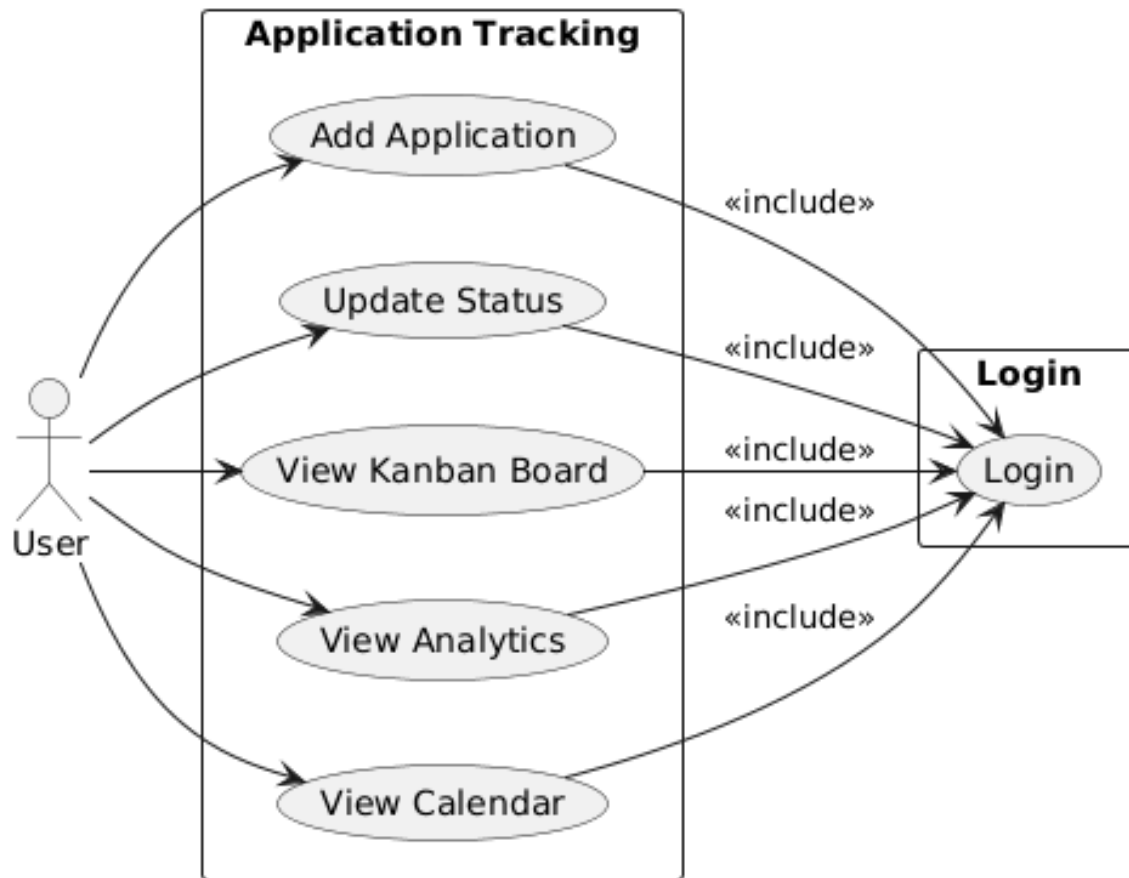


FIGURE 3.4 – Cas d'utilisation — Suivi des candidatures

Offres d'emploi et notifications L'utilisateur peut parcourir les offres d'emploi, déclencher un crawling en direct, configurer ses notifications, planifier des rappels et programmer des envois d'emails.

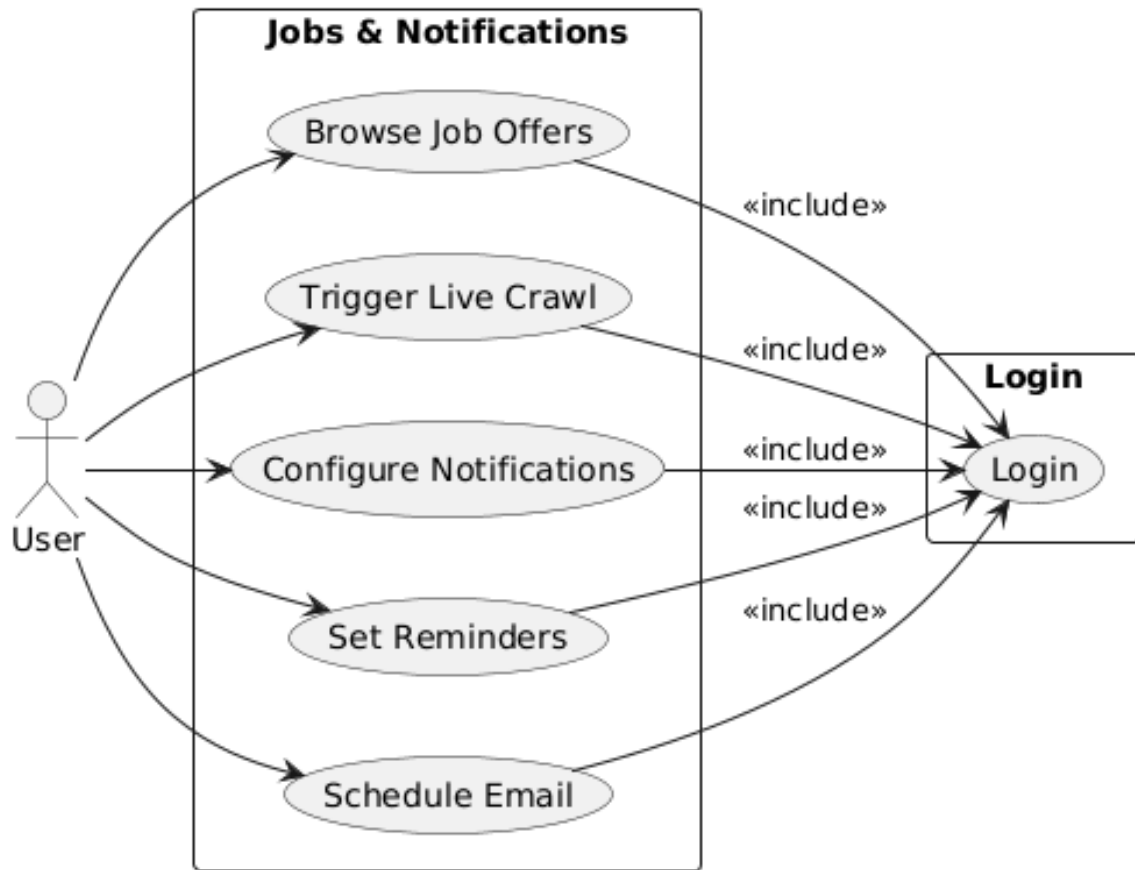


FIGURE 3.5 – Cas d’utilisation — Offres d’emploi et notifications

3.6.2 Diagramme de classes

Le diagramme de classes ci-dessous illustre les relations entre les entités principales du système, organisées par base de données. Chaque package coloré représente un microservice et sa base de données associée. Les relations montrent les dépendances inter-services, comme les références de l'utilisateur vers les contenus, candidatures et CV, ou les liens entre une candidature et une version de CV.

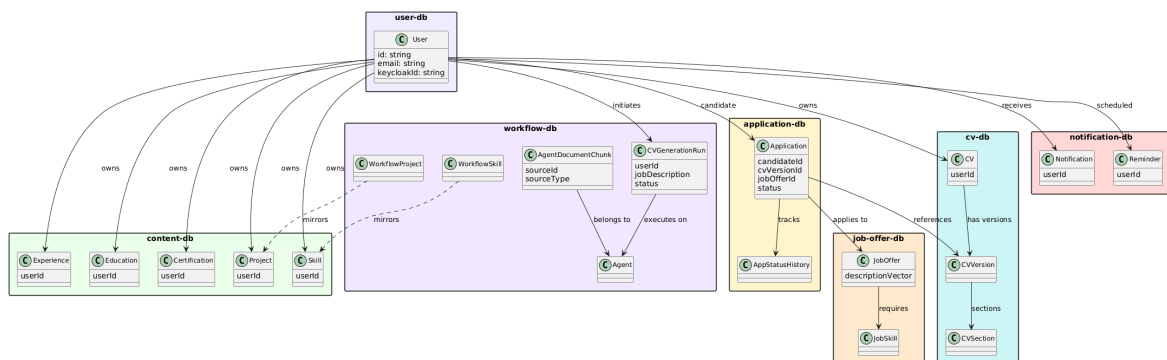


FIGURE 3.6 – Diagramme de classes — Relations inter-services

3.6.3 Diagramme de séquence — Flux d'authentification

Le diagramme de séquence ci-dessous illustre le flux d'authentification via Keycloak avec le protocole OpenID Connect (OIDC). L'utilisateur est redirigé vers Keycloak pour s'authentifier, reçoit un code d'autorisation, puis l'API Gateway échange ce code contre des jetons JWT. Un cookie de session (`cv_session`) est créé pour maintenir la session. Lors de l'inscription, un événement `user.registered` est publié sur Kafka pour notifier les autres services.

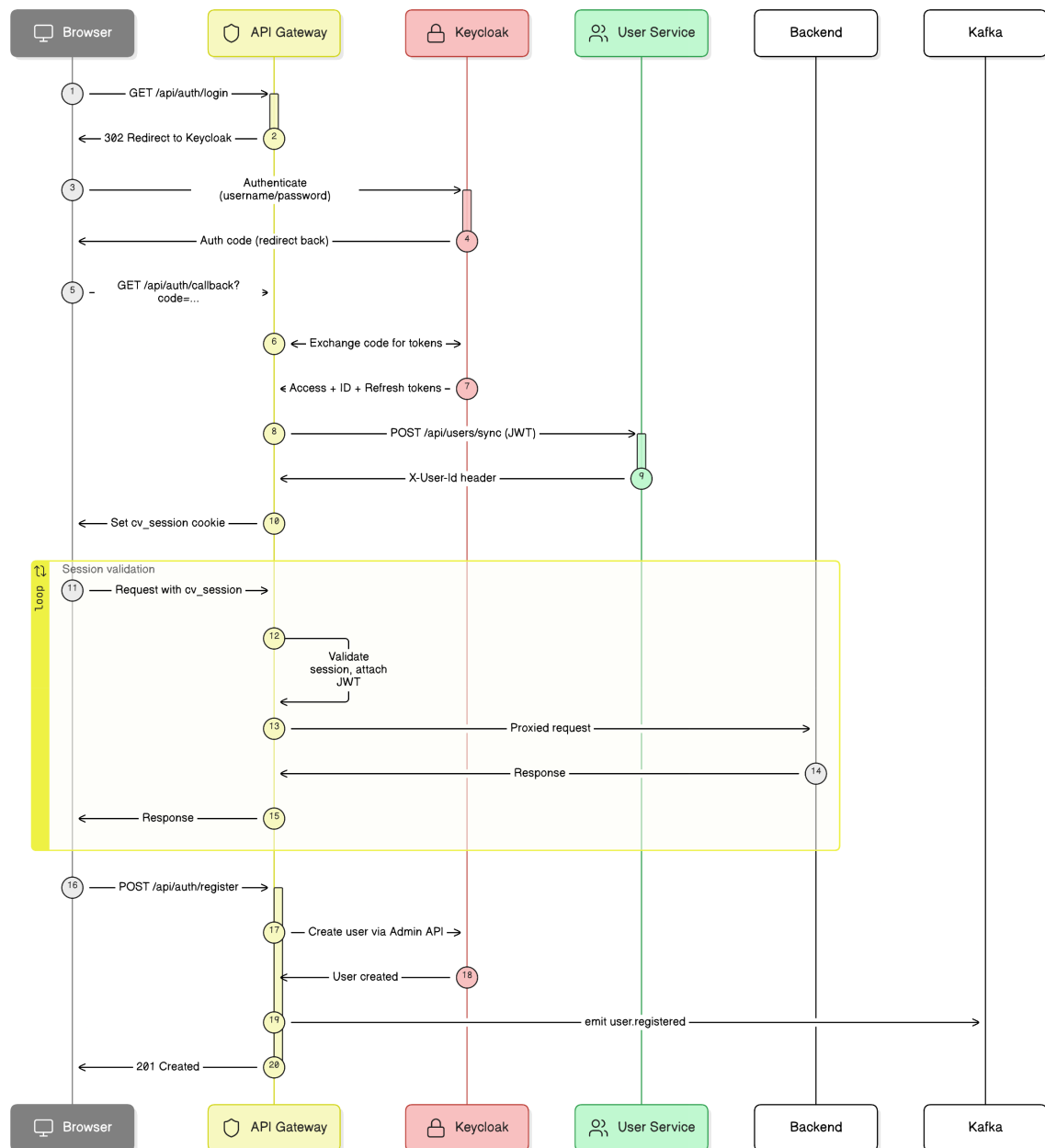


FIGURE 3.7 – Diagramme de séquence — Flux d'authentification OIDC

3.6.4 Modèle relationnel de la base de données

L’architecture microservices du projet implique l’utilisation de plusieurs bases de données PostgreSQL 16, une par service. Chaque base de données est spécialisée pour les besoins de son service et réside dans un conteneur Docker dédié. L’extension pg-vector est utilisée pour le support des recherches vectorielles dans les bases concernées par le pipeline IA.

Base Utilisateurs (user-db — port 5433)

La base utilisateurs centralise les informations personnelles et professionnelles des candidats.

- **users** : Table principale contenant le profil complet de chaque utilisateur. Les attributs incluent **keycloakId** (identifiant de synchronisation avec Keycloak), **headline** (titre professionnel), **desiredSalaryMin/desiredSalaryMax** (prétentions salariales), **requiresVisaSponsorship**, **noticePeriod** (préavis), **employmentTypes** (types d’emploi souhaités), **remotePreference** (préférence de télétravail), **willingToRelocate** (mobilité géographique). Des colonnes JSON permettent de stocker des données flexibles : **preferencesJson** et **aiProfileDataJson** (profil généré par l’IA).

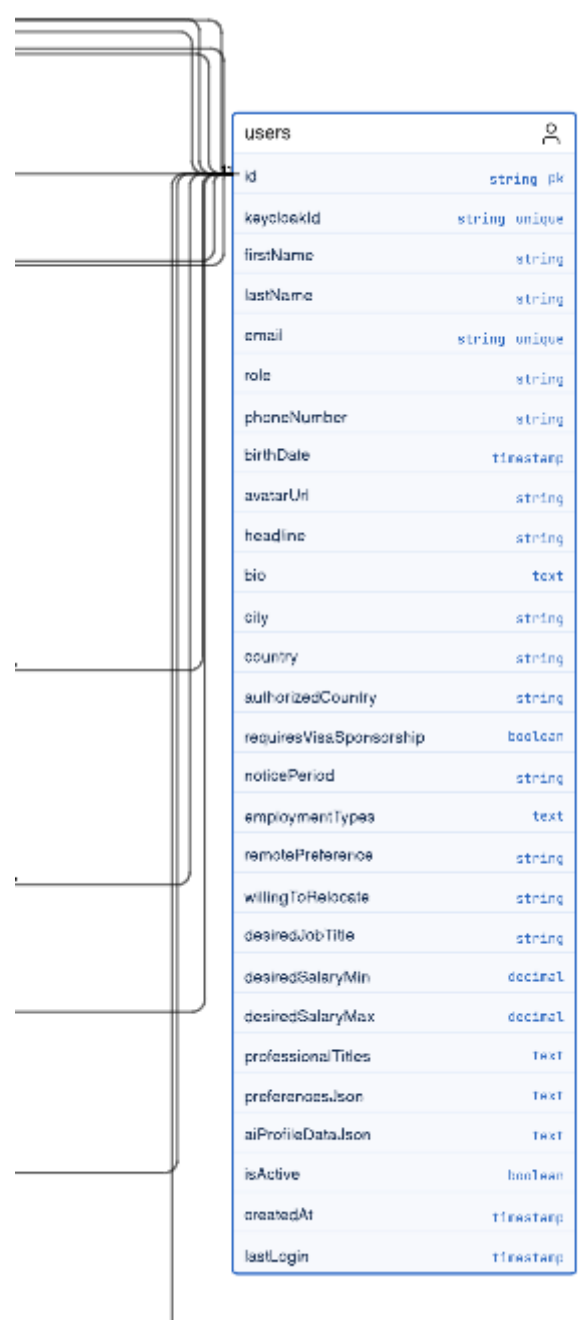


FIGURE 3.8 – Modèle relationnel — Base Utilisateurs

Base Contenu Utilisateur (content-db — port 5434)

Cette base gère l'ensemble du portfolio de l'utilisateur. Chaque table est reliée à l'utilisateur via une clé étrangère `userId`.

- **projects** : Projets réalisés. L'attribut `skillsJson` stocke les compétences associées sous forme JSON. `repositoryUrl` et `demoUrl` pointent vers les sources et démonstrations.
- **skills** : Compétences avec `category` (langage, framework, outil), `level` (débutant, intermédiaire, avancé, expert) et `yearsOfExperience`.

- **experiences** : Expériences professionnelles avec **company** et **status** (en cours, terminée).
- **educations** : Formations avec **degreeType** (licence, master, doctorat), **fieldOfStudy** et **specialization**. **diplomaFileUrl** stocke le lien vers le fichier du diplôme.
- **certifications** : Certifications avec **issuingOrganization** et **credentialUrl**.
- **languages** : Langues avec **level** (A1 à C2).
- **interests** : Centres d'intérêt.
- **hackathons** : Participations à des hackathons avec **role**, **result** et **description**.
- **socialLinks** : Liens vers les réseaux sociaux (**platform**, **url**).
- **cvProfiles** : Profils de CV avec **summary** (résumé personnel) et **title**.
- **academicActivities** : Activités académiques (projets scolaires, publications) avec période.



FIGURE 3.9 – Modèle relationnel — Base Contenu Utilisateur

Base Candidatures (application-db — port 5436)

Cette base assure le suivi des candidatures des utilisateurs.

- **applications** : Candidatures avec **candidateId**, **cvVersionId** et **jobOfferId** comme clés étrangères. **companyName** et **positionTitle** décrivent le poste visé.

`status` peut prendre les valeurs : `saved`, `applied`, `interview`, `offer`, `rejected`. `offerSource` indique la provenance de l'offre. `isSaved` permet de marquer une candidature comme favorite.

- `applicationStatusHistory` : Historique des changements de statut. `oldStatus` et `newStatus` tracent chaque transition. `changedBy` enregistre l'acteur (utilisateur ou système) et `comment` permet d'ajouter une note.
- `applicationConfigurations` : Préférences utilisateur pour le module. `showReminders` active les rappels, `selectedStatuses` filtre les statuts affichés.

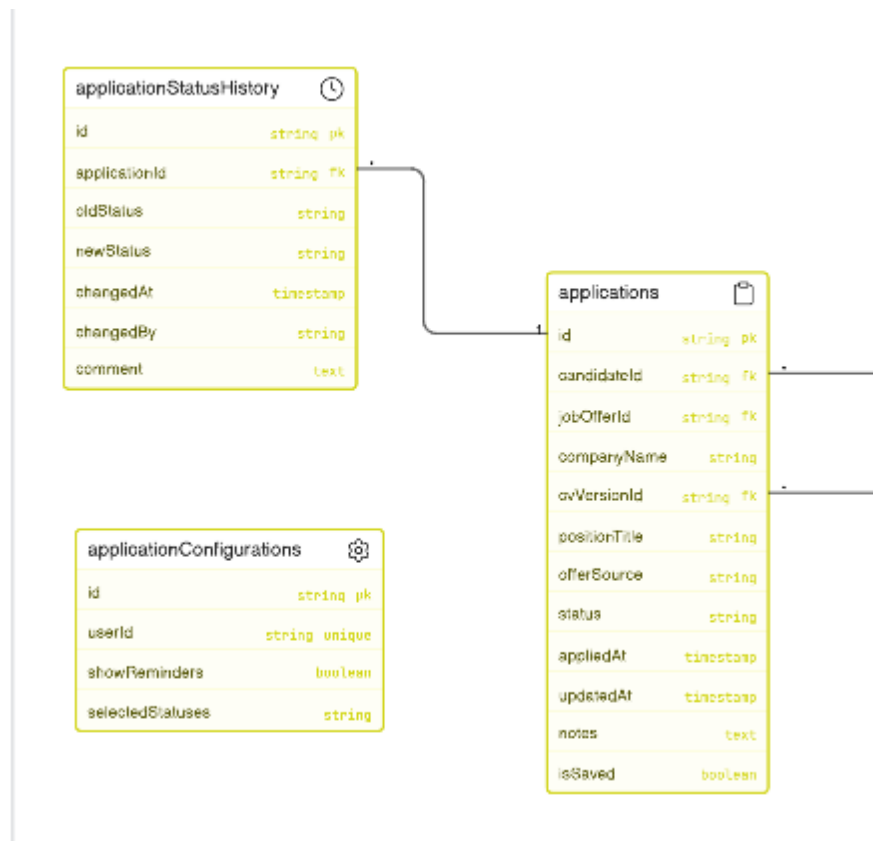


FIGURE 3.10 – Modèle relationnel — Base Candidatures

Base Workflow (workflow-db — port 5435)

La base workflow est la plus complexe. Elle pilote le pipeline de génération de CV et utilise pgvector pour les recherches vectorielles.

- `workflows` : Définition des pipelines. `definitionJson` contient la configuration du workflow au format JSON. `isActive` active ou désactive le pipeline.
- `cvGenerationRuns` : Traces d'exécution de chaque génération de CV. `jobDescription` stocke le texte brut de l'offre. `status` suit l'état (`pending`, `running`, `completed`, `failed`, `cancelled`). `stepStatuses` est un champ JSON contenant l'état de chaque étape. Les résultats des agents sont stockés dans des colonnes dédiées : `extractionResult`, `searchResult`, `renderResult`, `optimizationResult`, `deliveryResult`.

- **agents** : Configuration des agents IA. **agentId** est un identifiant unique (ex : job-extractor). **sortOrder** détermine l'ordre d'exécution dans le pipeline. **backgroundGradient** est utilisé pour l'affichage dans l'interface.
- **agentDocumentChunks** : Stockage des documents vectorisés pour le RAG. **sourceId** et **sourceType** identifient l'entité source. **embedding** est un vecteur pgvector utilisé pour la recherche sémantique. **searchVector** est un tsvector pour la recherche full-text.
- **jobExtractions** : Extractions d'offres d'emploi. **inputType** distingue le texte saisi de l'URL. **confidence** est un score de confiance de l'extraction. **outputJson** contient le résultat structuré.
- **workflowProjects**, **workflowSkills**, **workflowExperiences** : Entités miroir du contenu utilisateur, optimisées pour la recherche vectorielle avec des champs **embedding** et **aiSummaryJson**.



FIGURE 3.11 – Modèle relationnel — Base Workflow

Base Offres d'Emploi (job-offer-db — port 5437)

Cette base stocke les offres d'emploi collectées par l'agent de crawling et enrichies par l'agent d'extraction.

- **jobOffers** : Offres d'emploi. **jobHash** est un hash unique de l'offre pour éviter les doublons lors du crawling. **sourceUrl** est l'URL d'origine. **rawDescription** stocke la description brute. **descriptionVector** est un vecteur pgvector pour le matching avec le profil utilisateur. **locationType** distingue **remote**, **onsite** et **hybrid**. **employmentType** (**full-time**, **part-time**, **contract**) et **seniorityLevel**

(junior, mid, senior, lead) qualifie le poste.

- **jobSkills** : Compétences requises pour le poste. **type** distingue les compétences **required** et **preferred**. **isMandatory** indique si la compétence est obligatoire.
- **jobResponsibilities** : Liste des responsabilités du poste.
- **jobBenefits** : Avantages proposés par l'entreprise.
- **searchCaches** : Cache des recherches de crawling. **keyword** et **location** définissent la recherche. **status** suit l'avancement. **expectedCount** et **processedCount** permettent de suivre le nombre d'offres traitées.
- **userQuotas** : Quotas de crawling par utilisateur. **lastCrawlDate** enregistre la dernière date d'utilisation.

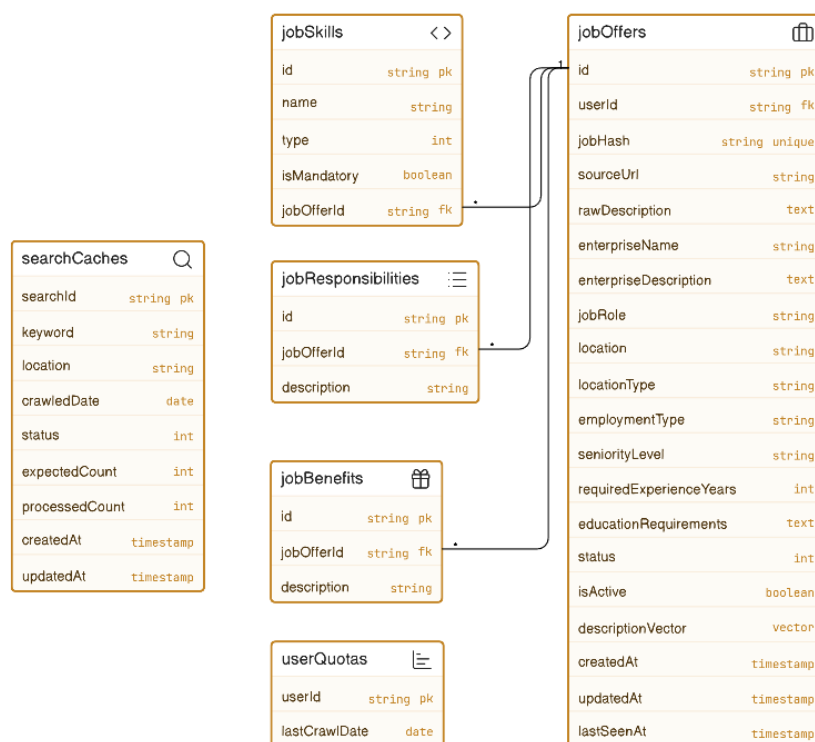


FIGURE 3.12 – Modèle relationnel — Base Offres d'Emploi

Base CV (cv-db — port 5439)

La base CV gère les curriculum vitae et leurs versions successives.

- **cvs** : CV avec **templateId** (identifiant du template utilisé) et **isActive** (CV actif par défaut).
- **cvVersions** : Versions d'un CV. **versionNumber** est un numéro incrémental. **contentJson** stocke le contenu complet du CV au format JSON structuré. **fileUrl** et **pdfUrl** pointent respectivement vers le fichier source et le PDF généré stockés dans MinIO.
- **cvSections** : Sections d'une version de CV. **sectionType** définit le type (**education**,

experience, skills, projects, certifications, languages). `contentJson` contient les données de la section. `displayOrder` détermine l'ordre d'affichage.

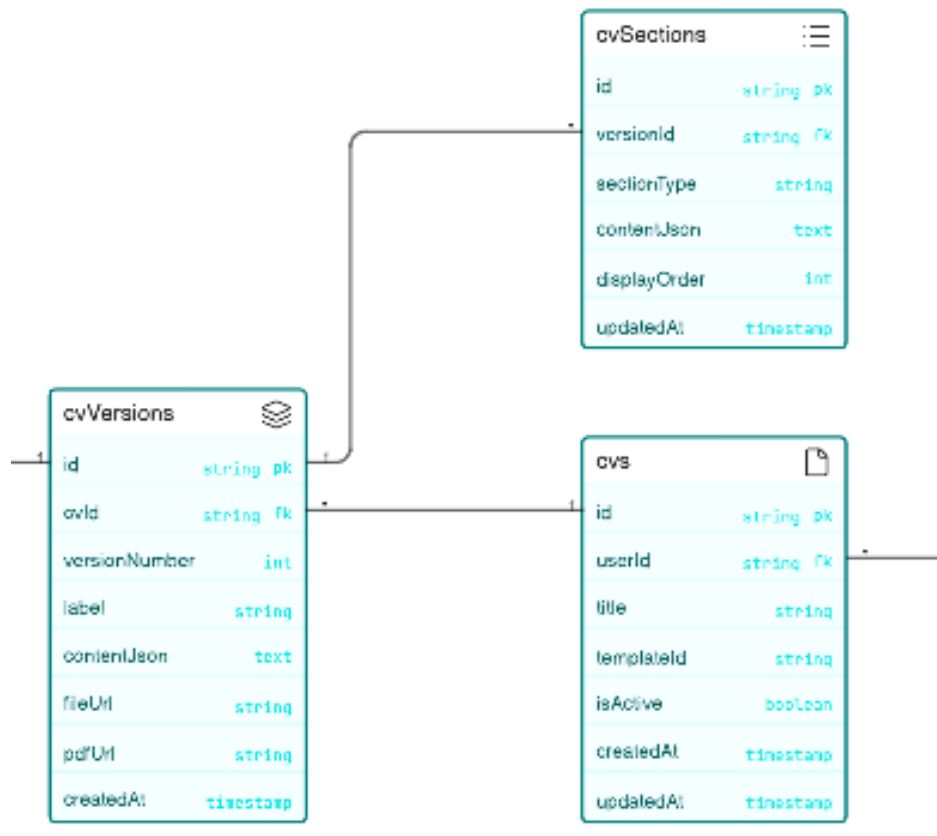


FIGURE 3.13 – Modèle relationnel — Base CV

Base Notifications (notification-db — port 5438)

Cette base gère l'ensemble du système de notifications, de rappels et de messagerie.

- **notifications** : Notifications in-app et email. `type` définit la catégorie (`application_update`, `cv_ready`, `reminder`, `welcome`). `channel` détermine le mode (`email`, `in-app`). `isRead` et `isSent` tracent l'état. `errorMessage` enregistre les échecs d'envoi.
- **notificationPreferences** : Préférences de notification par utilisateur. `enableEmail` et `enableInApp` activent les canaux. `applicationUpdates`, `cvUpdates`, `reminders`, `weeklyDigest` activent les catégories. `defaultReminderDaysBefore` définit le délai de rappel par défaut.
- **reminders** : Rappels planifiés. `eventDate` est la date de l'événement, `reminderAt` est la date du rappel. `reminderOffset` définit le décalage (ex : 1d, 2h). `status` suit l'état (`pending`, `sent`, `cancelled`).
- **contacts** : Carnet d'adresses. `source` indique la provenance (`manuel`, `import Gmail`). `isFavorite` permet de marquer les contacts favoris. `avatarBase64`

stocke l’avatar encodé.

- **emailMessages** : Historique des emails envoyés. **provider** indique le fournisseur (MailKit, Gmail API). **status** suit la délivrabilité. **contactId** relie l’email à un contact. **scheduleId** permet de lier à une campagne planifiée.
- **emailSchedules** : Campagnes d’emails planifiées. **cronExpression** définit la récurrence. **recipientIds** stocke les destinataires. **isActive** active ou désactive la campagne.
- **gmailConnections** : Connexions Gmail des utilisateurs. **encryptedAccessToken** et **encryptedRefreshToken** stockent les jetons OAuth chiffrés. **isRevoked** indique si la connexion a été révoquée.



FIGURE 3.14 – Modèle relationnel — Base Notifications

3.7 Conclusion

Ce chapitre a présenté l’analyse complète du système **Propel**, incluant l’étude de l’existant, les besoins fonctionnels et non fonctionnels, l’identification des acteurs et la modélisation. Le chapitre suivant détaillera la réalisation technique du projet.

Chapitre 4

Réalisation

4.1 Introduction

Ce chapitre présente la réalisation technique du projet **Propel**. Nous détaillons l'architecture physique et logicielle, les outils utilisés, le cycle DevSecOps mis en place, la gestion du code source, l'implémentation des différentes couches de l'application, ainsi que le déploiement.

4.2 Étude technique

4.2.1 Architecture physique

Infrastructure matérielle

Le projet s'appuie sur une infrastructure locale déployée sur des machines virtuelles VMware Workstation, provisionnées via Terraform. Chaque membre de l'équipe est responsable de VMs dédiées selon la répartition des services.

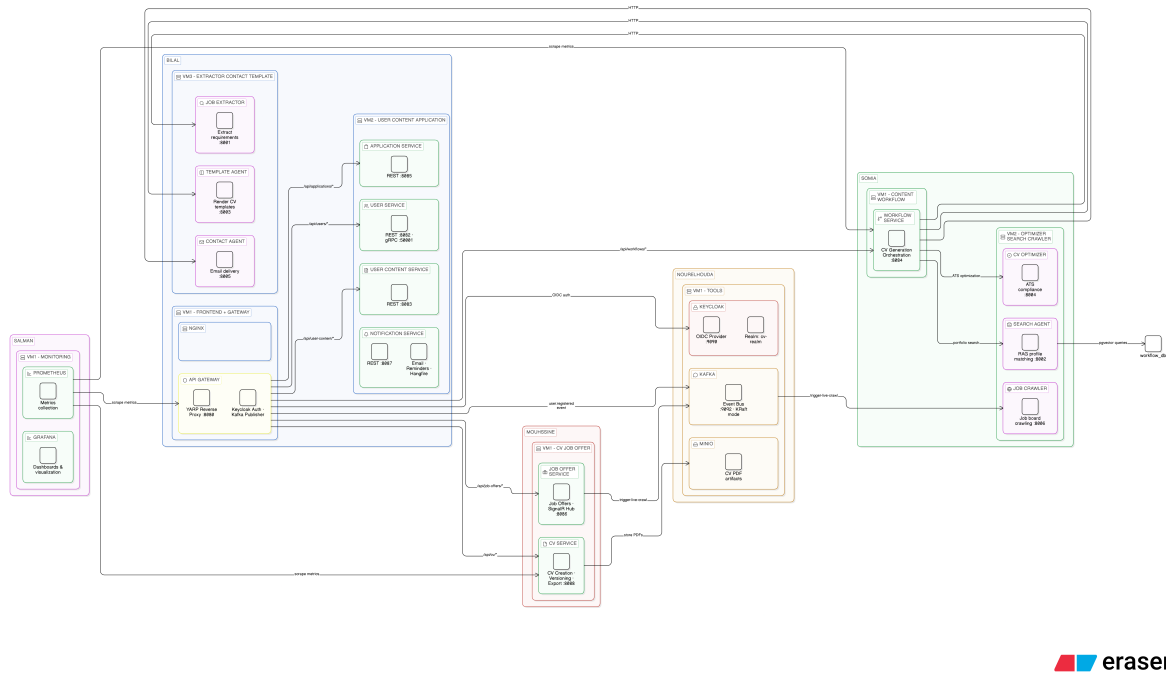


FIGURE 4.1 – Architecture de déploiement locale — Répartition des VMs par membre

La répartition des VMs est la suivante :

- **Bilal (3 VMs) :** Frontend + Gateway, Services applicatifs (User, Content, Application, Notification), Agents IA (Extractor, Contact, Template).
- **Somia (2 VMs) :** Service Workflow avec base pgvector, Agents IA (Optimizer, Search, Crawler).
- **Nourelhouda (1 VM) :** Infrastructure transverse (Keycloak, Kafka, MinIO).
- **Mouhssine (1 VM) :** Service CV et Service Offres d'Emploi.
- **Salman (1 VM) :** Monitoring (Prometheus, Grafana).

Chaque VM dispose de 2 CPU et 2 Go de RAM, et les services communiquent entre VMs via le réseau partagé Docker.

Infrastructure cloud

L'infrastructure cloud de **Propel** est déployée sur **Amazon Web Services (AWS)**, dans la région **us-east-1**. Elle repose sur deux outils complémentaires : **Terraform** pour le provisionnement de l'infrastructure en tant que code (**IaC**) et **Ansible** pour l'automatisation du déploiement applicatif. Le tableau suivant résume les principaux services AWS utilisés :

TABLE 4.1 – Services AWS utilisés dans l’infrastructure cloud

Catégorie	Service AWS	Rôle
Réseau	VPC, Subnets, NAT Gateway	Isolement réseau, sous-réseaux publics et privés
Exposition	Application Load Balancer	Point d’entrée unique, routage HTTP vers les services
Calcul	Amazon ECS (EC2), Auto Scaling Group	Orchestration des conteneurs sur instances EC2
Base de données	Amazon RDS PostgreSQL 16	Base de données managée, hautement disponible
Registre d’images	Amazon ECR	Stockage des images Docker des services
Secrets	AWS SSM Parameter Store	Gestion sécurisée des secrets et variables d’environnement
Logs	Amazon CloudWatch	Centralisation des logs, rétention 7 jours
Sécurité	IAM, Security Groups	Contrôle d’accès aux ressources AWS

Réseau et exposition L’infrastructure s’appuie sur un **VPC dédié** (CIDR : 10.20.0.0/16) structuré en sous-réseaux publics et privés répartis sur deux zones de disponibilité. Un **NAT Gateway** placé dans le sous-réseau public permet aux instances privées d’accéder à Internet en sortie sans être directement exposées.

L’**Application Load Balancer (ALB)** constitue le point d’entrée unique de l’application, écoutant sur le port 80 et acheminant les requêtes selon des règles de chemin :

- /api/* → Passerelle [API](#) (YARP)
- /realms/*, /resources/*, /admin/* → Keycloak (authentification)
- Tout le reste → Frontend Angular (Nginx)

Calcul — Amazon ECS sur EC2 L’orchestration des conteneurs est assurée par **Amazon ECS en mode EC2**, avec un cluster adossé à un **Auto Scaling Group** d’instances `m7i-flex.large` (2 vCPU, 8 Go de RAM, minimum 1 — maximum 3 instances). Le mode *ENI trunking* permet d’héberger jusqu’à 10 tâches par instance.

Le cluster héberge **18 services** au total : 15 services applicatifs (frontend, passerelle, 7 microservices .NET, 6 agents IA Python) et 3 services d’infrastructure (Kafka, Keycloak, MinIO). La communication inter-services est assurée par **ECS Service Connect**, qui fournit un **DNS** interne privé (`cvgen.local`) permettant aux services de se référencer par nom (ex. : `kafka:9092`, `minio:9000`, `user-service:8082`).

Base de données — Amazon RDS La persistance des données est assurée par une instance **Amazon RDS PostgreSQL 16** (classe `db.t3.micro`), déployée en sous-réseau privé et inaccessible publiquement. Elle héberge **8 bases de données logiques** correspondant aux 7 microservices .NET et à Keycloak. L’extension **pgvector** est activée sur les bases `workflow_db` et `cv_db` pour la recherche vectorielle utilisée par les agents **IA**. L’accès depuis l’extérieur lors des opérations de déploiement est sécurisé via un tunnel **AWS Systems Manager (SSM)**, sans ouverture de port SSH.

Registre d’images et stockage objet Chaque service dispose de son propre dépôt dans **Amazon ECR** (Elastic Container Registry), soit **16 dépôts** au total. Les images sont construites localement et poussées vers ECR lors de la phase de déploiement via Ansible.

Le stockage des fichiers **PDF** générés et des templates CV est assuré par **MinIO**, solution de stockage objet compatible S3, déployée comme conteneur ECS avec deux buckets : `cv-pdfs` et `cv-templates`.

Sécurité et gestion des secrets La sécurité de l’infrastructure repose sur trois mécanismes complémentaires :

- **IAM (Identity and Access Management)** : Chaque service ECS dispose d’un rôle IAM minimal avec uniquement les permissions nécessaires (ECR pull, SSM read, CloudWatch Logs). Les instances EC2 disposent d’un rôle dédié pour ECS et SSM Session Manager.
- **AWS SSM Parameter Store** : L’ensemble des secrets (chaînes de connexion, clés API, mots de passe) est stocké sous forme de paramètres chiffrés (**SecureString**). Terraform crée les paramètres avec des valeurs provisoires, puis Ansible les remplace par les valeurs réelles lors du déploiement.
- **Security Groups** : Des groupes de sécurité distincts isolent les différentes couches : ALB (port 80 public), instances ECS (ports internes uniquement) et RDS (port 5432 accessible depuis ECS uniquement).

Provisionnement IaC — Terraform L'ensemble de l'infrastructure AWS est défini sous forme de code avec **Terraform** (provider AWS $\sim > 5.0$). Les ressources sont organisées en fichiers thématiques : `network.tf`, `ecs-cluster.tf`, `rds.tf`, `alb.tf`, `iam.tf`, `ssm.tf`, `ecr.tf` et `services.tf`. Un **module réutilisable** (`modules/service/`) factorise la définition des 15 services ECS applicatifs afin d'éviter la duplication de configuration.

Automatisation du déploiement — Ansible Le déploiement applicatif est automatisé via **Ansible** (≥ 2.14), organisé en 5 rôles exécutés séquentiellement :

1. **images** : Construction et poussée des 16 images Docker vers ECR.
2. **database** : Création des 8 bases PostgreSQL et activation de l'extension `pgvector` via tunnel SSM.
3. **secrets_overlay** : Écriture des valeurs réelles dans SSM Parameter Store (clés API, mots de passe, DNS de l'ALB).
4. **deploy** : Redémarrage forcé des services ECS (`force-new-deployment`) pour récupérer les nouvelles images.
5. **smoketest** : Vérification automatique des endpoints critiques après déploiement.

Environnement de déploiement

L'ensemble de l'application est conteneurisé à l'aide de **Docker**. Chaque service possède son propre Dockerfile avec une construction multi-stage optimisée. L'orchestration est assurée par **Docker Compose** au niveau racine du projet, qui inclut trois fichiers compose spécialisés :

- **backend/docker-compose.yml** : Contient les 7 microservices backend .NET, les 8 bases de données PostgreSQL, Kafka, et MinIO.
- **ai_agents/docker-compose.yml** : Contient le pipeline de 6 agents IA en Python avec leur orchestrateur.
- **api_gateway/docker-compose.yml** : Contient la passerelle API (YARP) et Keycloak.

Tous les services partagent un réseau Docker commun (`cv-network`) pour la communication inter-services.

4.2.2 Architecture logicielle

L'architecture logicielle de **Propel** est une architecture en **microservices** avec communication événementielle. Le schéma ci-dessous illustre l'architecture globale du système.

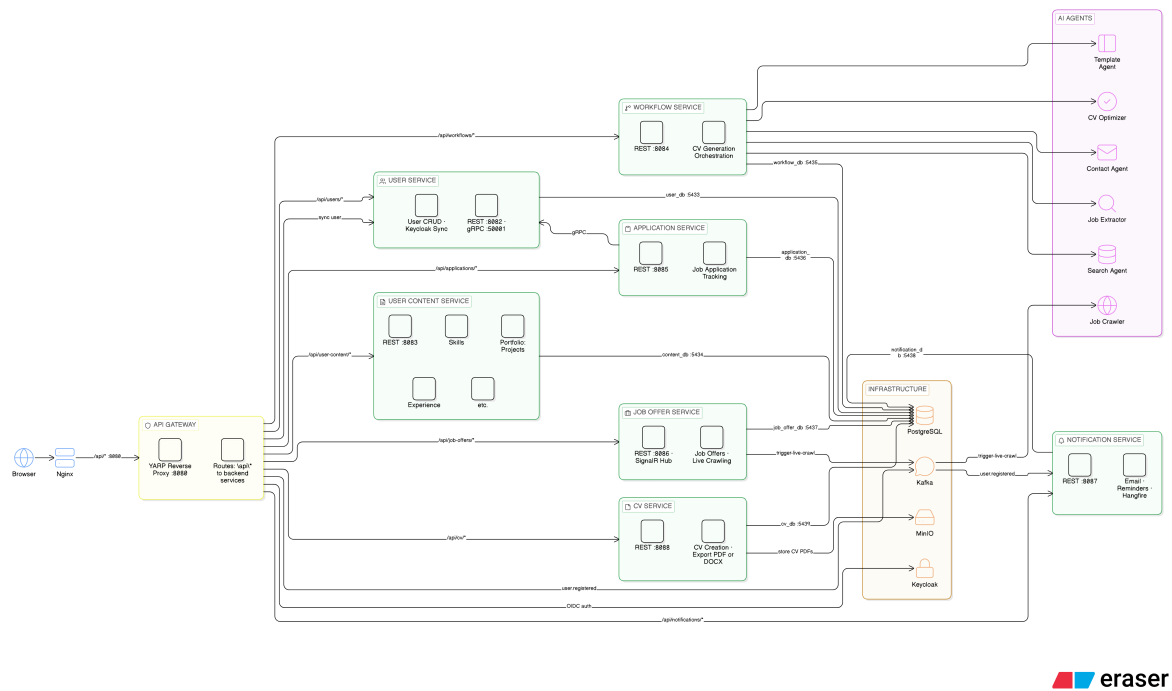


FIGURE 4.2 – Architecture logicielle du système

L'architecture se compose des couches suivantes :

- **Frontend (Angular)** : Application monopage (SPA) hébergée par Nginx, consommant l'API via la passerelle.
- **Passerelle API (YARP)** : Proxy inverse .NET assurant le routage, l'authentification et la gestion des sessions. Publie les événements de connexion vers Kafka.
- **Backend (.NET)** : 7 microservices indépendants, chacun avec sa propre base de données PostgreSQL. Ils communiquent via HTTP, gRPC et Kafka.
- **Agents IA (Python)** : Pipeline de 6 agents FastAPI utilisant LangChain et LangGraph pour le traitement intelligent des CV.
- **Messagerie (Kafka)** : Système de messagerie asynchrone en mode KRaft (sans Zookeeper) pour la communication événementielle entre services.
- **Stockage objet (MinIO)** : Stockage S3-compatible pour les fichiers PDF générés.
- **Authentification (Keycloak)** : Serveur d'authentification centralisé avec support OAuth2, OpenID Connect et fédération d'identités (Google, GitHub).

4.2.3 Présentation des outils utilisés

Outils de développement

Le tableau suivant récapitule les principaux outils et technologies utilisés pour le développement :

TABLE 4.2 – Outils de développement

Couche	Technologies
Frontend	Angular 21, TypeScript, RxJS, Signals, SCSS, Vitest, Cypress
Backend	ASP.NET Core 10, Entity Framework Core, gRPC, YARP, Hangfire, MailKit
Agents IA	Python, FastAPI, LangChain, LangGraph, LangChain-Groq, Pydantic
Bases de données	PostgreSQL 16 avec extension pgvector
Authentification	Keycloak 26, OpenID Connect, JWT
Messagerie	Apache Kafka (KRaft mode)
Stockage	MinIO (API S3)
Conteneurisation	Docker, Docker Compose

Outils d’intégration et de déploiement continu (CI/CD)

TABLE 4.3 – Outils CI/CD et qualité

Outil	Rôle
GitHub Actions	Pipeline CI/CD (6 phases)
SonarCloud	Analyse statique de code et quality gates
Trivy	Scan de vulnérabilités (système de fichiers et images Docker)
OWASP ZAP	Tests de sécurité DAST (Dynamic Application Security Testing)
Docker	Conteneurisation multi-stage
GitHub Container Registry	Registre d’images Docker
DockerHub	Registre d’images secondaire

4.3 Cycle DevSecOps

4.3.1 Présentation et importance dans le projet

Le cycle DevSecOps intègre la sécurité à chaque étape du cycle de développement logiciel. Dans le cadre de **Propel**, cette approche est essentielle étant donné la nature sensible des données traitées (informations personnelles des utilisateurs, CV, données de connexion).

L'adoption du cycle DevSecOps nous permet de :

- Détecter les vulnérabilités tôt dans le cycle de développement.
- Automatiser les contrôles de sécurité.
- Garantir la conformité aux bonnes pratiques.
- Assurer une traçabilité complète des modifications.

4.3.2 Outils DevSecOps utilisés

- **SonarCloud** : Analyse statique de code avec quality gates pour les projets Angular, .NET et Python. Des seuils de qualité (couverture de code, dette technique, duplication) doivent être respectés.
- **Trivy** : Scan de vulnérabilités sur le système de fichiers lors des phases statiques et sur les images Docker construites. Détection des dépendances obsolètes et des failles de sécurité connues.
- **OWASP ZAP** : Tests DAST (Dynamic Application Security Testing) en environnement d'intégration pour détecter les vulnérabilités applicatives (XSS, injection SQL, etc.).

4.3.3 Application du cycle DevSecOps dans le projet

Le cycle DevSecOps est appliqué à travers le pipeline CI/CD en six phases :

1. **Phase 1 - Static Checks** : Validation des fichiers Docker Compose, scan Trivy filesystem, analyse SonarCloud pour le frontend, le backend et les agents IA.
2. **Phase 2 - Component Tests** : Exécution des tests unitaires (.NET avec xUnit, Angular avec Vitest, Python avec pytest) et génération des rapports de couverture.
3. **Phase 3 - Integration / E2E** : Tests d'intégration et tests E2E avec Cypress, suivis d'un scan de sécurité OWASP ZAP.
4. **Phase 4 - Build & Scan** : Construction des images Docker multi-stage, suivie d'un scan Trivy des images.
5. **Phase 5 - Registry Push** : Publication des images validées vers GitHub Container Registry et DockerHub.

6. **Phase 6 - Deploy** : Déploiement de l'application sur l'environnement cible.

4.3.4 Sécurité applicative

La sécurité applicative est assurée à plusieurs niveaux :

- **Authentification centralisée** : Keycloak avec OpenID Connect, support OAuth2 (Google, GitHub), gestion des sessions par cookies sécurisés JWT.
- **Protection des API** : Validation des tokens JWT, gestion des rôles et permissions.
- **Sécurisation des communications** : HTTPS, chiffrement des données sensibles.
- **Scan de vulnérabilités** : Automatisé via Trivy (images et dépendances) et OWASP ZAP (tests dynamiques).
- **Gestion des secrets** : Fichier `.env` avec variables d'environnement, exclusion des secrets du versionnement via `.gitignore`.

4.4 Gestion du code source

4.4.1 Utilisation de Git

Le code source du projet est versionné avec **Git** et hébergé sur **GitHub**. L'utilisation de Git permet un suivi précis de l'historique des modifications, une collaboration efficace entre les membres de l'équipe et une intégration fluide avec le pipeline CI/CD.

4.4.2 Organisation des branches

La stratégie de branches adoptée est une approche **Git Flow** simplifiée :

- **main** : Branche de production, protégée, ne reçoit que des merges validés.
- **develop** : Branche d'intégration continue pour le développement.
- **feature/*** : Branches de fonctionnalités créées à partir de **develop** et fusionnées via Pull Request.
- **hotfix/*** : Branches de correction urgente créées à partir de **main**.

FIGURE 4.3 – Organisation des branches Git

4.4.3 Gestion des commits et collaboration

La collaboration s'effectue via **Pull Requests** sur GitHub. Chaque PR est soumise à une revue de code par le chef d'équipe (Bilal Magguilej) avant d'être fusionnée. Le

pipeline CI/CD s'exécute automatiquement sur chaque PR pour valider la qualité et la sécurité du code proposé.

4.5 Implémentation

4.5.1 Développement Backend

Le backend est composé de 7 microservices .NET, chacun responsable d'un domaine métier spécifique :

- **User Service** : Gestion des utilisateurs, synchronisation avec Keycloak, communication gRPC.
- **User Content Service** : Gestion du portfolio (projets, compétences, expériences, formations, certifications, langues).
- **CV Service** : Création et versionnement des CV, gestion des templates, export PDF.
- **Workflow Service** : Orchestration de la génération de CV, appel aux agents IA.
- **Application Service** : Suivi des candidatures, communication gRPC avec User Service.
- **Job Offer Service** : Gestion des offres d'emploi, déclenchement du crawling, hub SignalR.
- **Notification Service** : Envoi d'emails (MailKit), rappels planifiés (Hangfire), consommateur Kafka.

La passerelle API (**YARP**) est un service .NET dédié qui agit comme point d'entrée unique. Elle gère l'authentification, le routage vers les microservices et la publication d'événements Kafka.

La communication événementielle est assurée par **Apache Kafka** en mode KRaft. Le diagramme ci-dessous illustre les principaux événements échangés entre les services via Kafka.

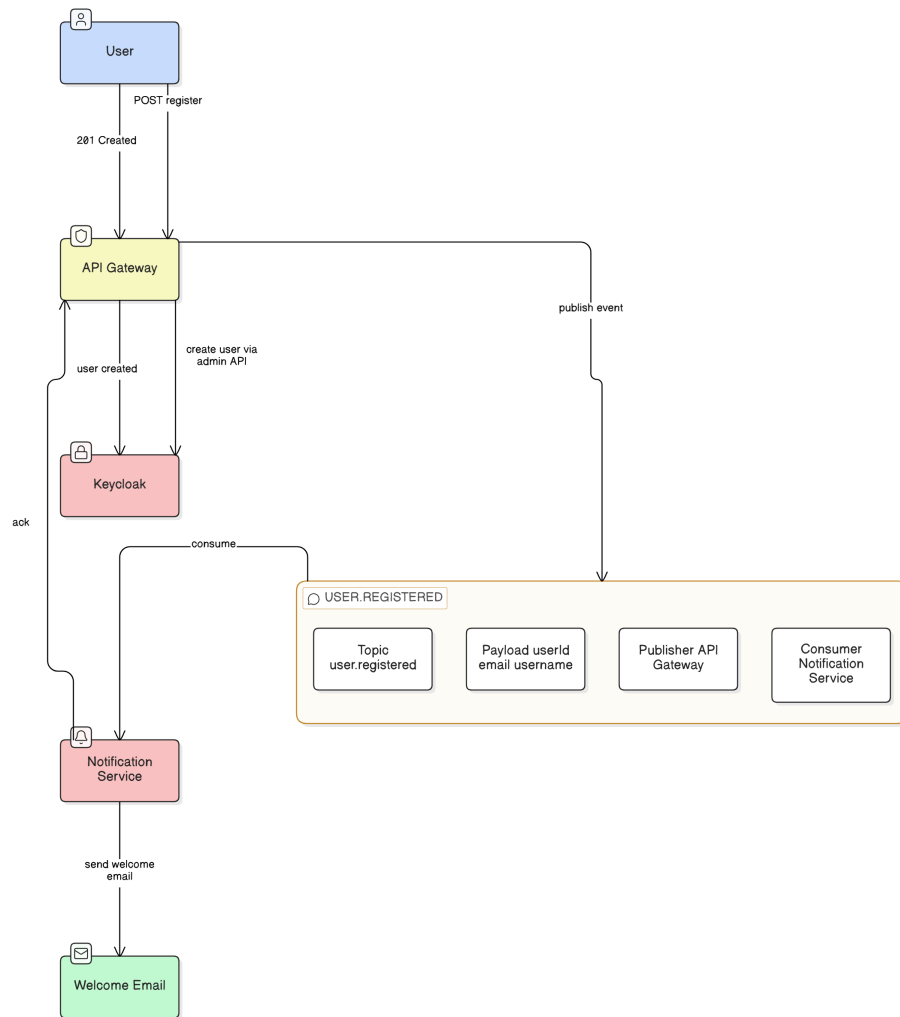


FIGURE 4.4 – Flux des événements Kafka — Publications et consommations

4.5.2 Pipeline de génération de CV

Le **Workflow Service** orchestre le pipeline de génération de CV, composé de 5 agents IA agissant en séquence. Chaque agent traite une étape spécifique du flux de travail.

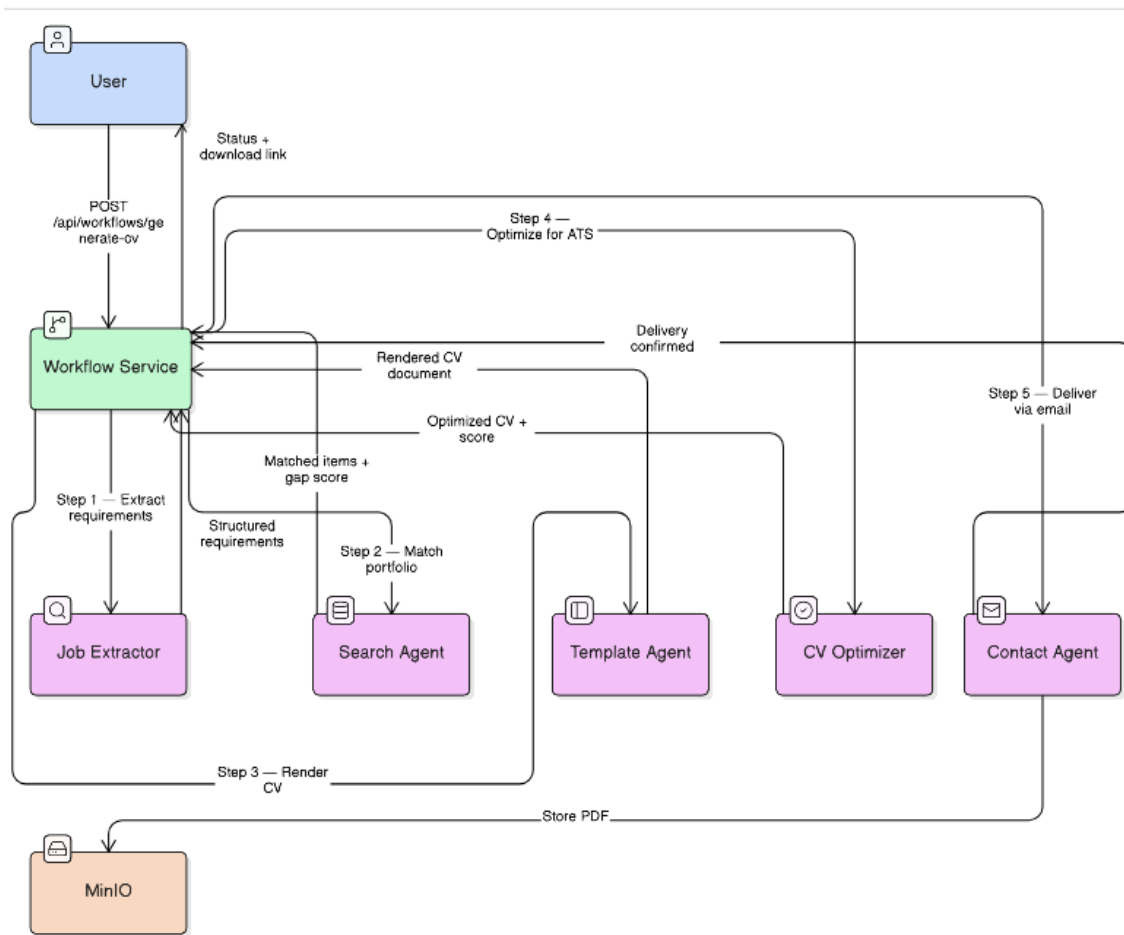


FIGURE 4.5 – Pipeline de génération de CV — De l’offre d’emploi à l’envoi par email

4.5.3 Développement Frontend

Le frontend est développé avec **Angular 21** en utilisant les **standalone components**, **RxJS** pour la gestion asynchrone et les **Signals** pour la réactivité. L’interface suit le design system **Propel** avec des couleurs en espace OKLCH.

Les principales fonctionnalités du frontend incluent :

- Pages de connexion et d’inscription (Keycloak).
- Tableau de bord avec statistiques et graphiques.

- Interface de génération de CV (saisie d’offre d’emploi).
- Vue Kanban pour le suivi des candidatures (drag-and-drop).
- Vue calendrier avec rappels.
- Gestion complète du portfolio.
- Vues liste et analytique des candidatures.

4.5.4 Base de données

L’architecture microservices implique l’utilisation de 8 bases de données **PostgreSQL 16** indépendantes, une par microservice, plus une pour Keycloak. Chaque base utilise l’extension **pgvector** pour le support des recherches vectorielles utilisées par l’agent de matching RAG.

4.6 Déploiement

4.6.1 Mise en place de l’environnement de développement

L’environnement de développement est entièrement conteneurisé. Un fichier **Makefile** à la racine du projet automatise les commandes courantes (démarrage des services, exécution des migrations, lancement des tests). Chaque sous-projet (backend, ai_agents) possède également son propre Makefile.

4.6.2 Dockerisation de l’application

L’ensemble de l’application est dockerisé avec des **Dockerfiles multi-stage** pour optimiser la taille des images de production. Au total, 18 Dockerfiles ont été créés couvrant l’ensemble des services.

Les images sont construites en deux étapes :

1. **Build** : Compilation et génération des artefacts.
2. **Runtime** : Image légère ne contenant que l’exécutable et ses dépendances.

4.6.3 Pipeline CI/CD

Le pipeline CI/CD est implémenté avec **GitHub Actions** et comprend 6 phases séquentielles. Chaque phase doit réussir pour passer à la suivante.

FIGURE 4.6 – Pipeline CI/CD

4.7 Présentation de l'application

4.7.1 Interface utilisateur

FIGURE 4.7 – Interface du tableau de bord

FIGURE 4.8 – Interface de génération de CV

FIGURE 4.9 – Tableau Kanban de suivi des candidatures

FIGURE 4.10 – Interface de gestion du portfolio

4.7.2 Fonctionnalités principales

Le système **Propel** offre les fonctionnalités principales suivantes :

- **Génération intelligente de CV** : Pipeline de 6 agents IA (extraction, matching RAG, template, optimisation ATS, contact) coordonnés par un orchestrateur.
- **Suivi de candidatures** : Vue Kanban avec drag-and-drop, vue liste, vue calendrier, graphiques analytiques.
- **Gestion de portfolio** : CRUD complet pour projets, compétences, expériences, formations, certifications, langues et réseaux sociaux.
- **Crawling d'offres d'emploi** : Agent IA dédié qui collecte automatiquement les offres depuis des plateformes externes.
- **Notifications temps réel** : SignalR pour les mises à jour en direct, Hangfire pour les rappels planifiés, emails via MailKit.

4.7.3 Démonstrations du fonctionnement

4.8 Assurance Qualité

L'assurance qualité du projet **Propel** repose sur une stratégie de tests à plusieurs niveaux : tests unitaires, tests d'intégration, tests bout en bout (E2E) et tests de performance. Cette approche garantit la fiabilité de chaque composant isolément, la cohérence des interactions entre services, la conformité des parcours utilisateur, et la tenue en charge du système sous conditions réelles.

Mohssine El Addaoui est responsable de cette couverture de tests, en charge de l'outillage Cypress pour les tests E2E et k6 pour les tests de performance.

4.8.1 Tests Unitaires

Les tests unitaires visent à vérifier le comportement de chaque unité de code de manière isolée, en remplaçant les dépendances externes par des mocks.

Tests unitaires des agents IA — pytest

L'agent de contact (`contact-agent`) dispose d'une suite de **43 tests pytest** répartis en quatre modules :

- **test_tools.py (19 tests)** : Teste les quatre outils LangGraph de l'agent de manière indépendante.
 - `extract_cv_text` — 8 cas : tri par ordre de section, liste vide, JSON invalide, sections sans contenu, présence de plusieurs sections, champs `order` manquants, contenu avec espaces, section unique.
 - `generate_email_subject` — 2 cas : retour d'un objet ligne d'objet, vérification que le prompt contient bien les informations de l'offre (LLM mocké via `unittest.mock`).
 - `generate_email_body` — 3 cas : retour du corps de l'email, présence de toutes les entrées dans le prompt, gestion d'un `hint` vide.
 - `send_email_with_cv` — 6 cas : envoi réussi sans PDF, identifiants manquants, échec d'authentification SMTP, envoi avec URL PDF valide, échec de téléchargement PDF, fichier PDF local introuvable (SMTP mocké via `smtplib`).
- **test_agent.py (7 tests)** : Teste l'orchestration LangGraph de la fonction `deliver_cv()`.
 - `deliver_cv()` — 4 cas : livraison réussie, livraison échouée, génération de l'ID de livraison lorsqu'absent, gestion d'une URL PDF nulle.
 - `_extract_tool_output()` — 3 cas : recherche d'un outil par nom, retour de N/A si introuvable, suppression des espaces.
- **test_schemas.py (12 tests)** : Valide les schémas Pydantic `ContactInput` et `ContactOutput`.
 - `ContactInput` — 8 cas : entrée valide, avec `cover_letter_hint`, avec URL PDF, champs obligatoires manquants (`job_title`, `recipient_email`, `optimized_cv`), sections CV vides, valeurs limites du score ATS (0, 100, -1, 101).
 - `ContactOutput` — 4 cas : sortie réussie, sortie échouée, génération automatique de `sent_at`, champ `delivery_id` manquant.
- **test_integration.py (5 tests)** : Teste l'endpoint FastAPI `/deliver` via le client ASGI (agent mocké).
 - 4 tests avec agent mocké : succès, échec, erreur de validation (422), et endpoint `/health`.

- 1 test live (marqué `@pytest.mark.live`, exécuté uniquement avec `-run-live`) : envoi réel d'un email de démonstration.

FIGURE 4.11 – Résultats de l'exécution des tests pytest — contact-agent

Tests unitaires du backend .NET — xUnit

Le projet `ApplicationService.Tests` contient **15 tests xUnit** organisés en trois classes, utilisant **NSubstitute** pour les mocks et **FluentValidation.TestHelper** pour les validateurs :

- **ApplicationsControllerTests (4 tests)** : Teste les cas aux limites du contrôleur REST.
 - `GetAll` retourne 200 avec la liste des candidatures.
 - `GetById` retourne 404 lorsque la candidature appartient à un autre utilisateur.
 - `Create` retourne 201 avec l'ID de candidat forcé.
 - `GetAll` retourne 401 lorsqu'aucun utilisateur n'est authentifié.
- **ApplicationServiceImplTests (5 tests)** : Teste la logique métier du service.
 - `GetAllAsync` retourne des résultats paginés correctement.
 - `CreateAsync` publie un événement Kafka à la création.
 - `UpdateStatusAsync` retourne `null` si la candidature est introuvable.
 - `DeleteAsync` retourne `false` si la candidature est introuvable.
 - `GetStatisticsAsync` calcule correctement les compteurs par statut.
- **CreateApplicationValidatorTests (6 tests)** : Teste les règles de validation `FluentValidation`.
 - Passage réussi lorsque tous les champs sont valides.
 - Échec si `CandidateId` est vide (GUID vide).
 - Échec si `CompanyName` est vide ou dépasse 200 caractères.
 - Échec si `PositionTitle` est vide ou dépasse 150 caractères.

La fixture `DatabaseFixture.cs` utilise **Testcontainers.PostgreSql** pour démarrer une vraie base de données PostgreSQL dans un conteneur Docker lors des tests nécessitant de vraies interactions avec la base.

FIGURE 4.12 – Résultats de l'exécution des tests xUnit — ApplicationService

Tests unitaires du frontend Angular — Vitest

Le frontend Angular comprend **5 fichiers de spécifications Vitest** couvrant les composants principaux :

- `app.spec.ts` — Initialisation du composant racine.

- `header.component.spec.ts` — Composant d'en-tête.
- `sidebar.component.spec.ts` — Composant de barre latérale.
- `home.component.spec.ts` — Page d'accueil.
- `login.component.spec.ts` — Page de connexion.

Ces tests vérifient la création et l'initialisation correcte de chaque composant Angular de manière isolée.

4.8.2 Tests d'Intégration

Les tests d'intégration vérifient la coopération entre plusieurs composants réels : couche HTTP, logique métier, base de données et messagerie.

Tests d'intégration des agents IA

Le module `test_integration.py` de l'agent de contact utilise le client ASGI de **FastAPI** (via `httpx.AsyncClient`) pour tester l'application complète sans démarrer de serveur HTTP réel :

- **TestDeliverEndpoint (4 tests)** : L'agent est mocké, mais la couche de routing FastAPI, la validation Pydantic et la gestion des codes HTTP sont entièrement réelles.
 - Succès de la livraison — réponse 200 avec `delivery_id` et `sent_at`.
 - Échec de la livraison — réponse 200 avec `success: false`.
 - Erreur de validation — réponse 422 sur payload invalide.
 - Endpoint `/health` — réponse 200 avec statut `ok`.
- **TestLiveEmail (1 test optionnel)** : Envoi réel d'un email en appelant le vrai LLM Groq et le vrai serveur SMTP, déclenché uniquement avec `pytest -run-live`.

Tests d'intégration du backend .NET

Le projet `ApplicationService.Tests` utilise **Testcontainers.PostgreSql** pour les scénarios nécessitant une vraie base de données. La classe `DatabaseFixture` démarre un conteneur PostgreSQL éphémère en début de suite de tests et le détruit à la fin. Cette approche garantit que les tests ne se contentent pas de mocker les requêtes mais valident réellement le comportement de **Entity Framework Core** face à PostgreSQL — migrations incluses.

FIGURE 4.13 – Résultats des tests d'intégration — Testcontainers + FastAPI Test-Client

4.8.3 Tests End-to-End (E2E) — Cypress

Les tests E2E simulent des scénarios utilisateurs complets à travers l'interface Angular, en interagissant avec la vraie API déployée sur une base de données de test dédiée.

Infrastructure et configuration

L'outillage E2E repose sur **Cypress** avec les éléments suivants :

- **Fixtures JSON** : `users.json`, `application.json`, `career.json`, `profile.json`, `reminder.json` — fournissent des données de test déterministes pour chaque scénario.
- **Commandes personnalisées** (`support/commands.ts`) :
 - `cy.login(email, password)` — envoie POST `/api/auth/login` et valide la session.
 - `cy.loginAsUser()` — utilise les identifiants de `users.json` pour se connecter rapidement avant chaque test.
 - `cy.disableNativeValidation()` — désactive la validation HTML5 native pour tester la validation applicative.
- **Interception réseau** : `cy.intercept()` alias les requêtes HTTP (`@loginReq`, `@createApp`, etc.) et `cy.wait()` valide les codes de réponse et les corps renvoyés par l'API.

Couverture des scénarios

La suite E2E totalise **24 tests** répartis en 6 fichiers de spécifications :

TABLE 4.4 – Fichiers de spécifications Cypress et couverture

Fichier	Tests	Parcours couverts
smoke.cy.ts	2	Chargement des pages / et /login, présence du branding
auth.cy.ts	10	Rendu de la page de connexion, bascule inscription/connexion, validation côté client, inscription (idempotente : 200 ou 409), connexion avec mauvais identifiants (401), connexion réussie et redirection vers /applications
application-create.cy.ts	2	Validation des champs obligatoires, création d'une candidature et navigation vers la page de détail
reminder-create.cy.ts	3	Ouverture du modal rappel, validation des champs manquants, création d'un rappel et affichage du bandeau de succès
my-career.cy.ts	4	Ajout d'une entrée par type : expériences, projets, compétences, formations
personal-info.cy.ts	3	Chargement de la page profil, mise à jour des informations et bandeau de succès, ajout d'un titre professionnel

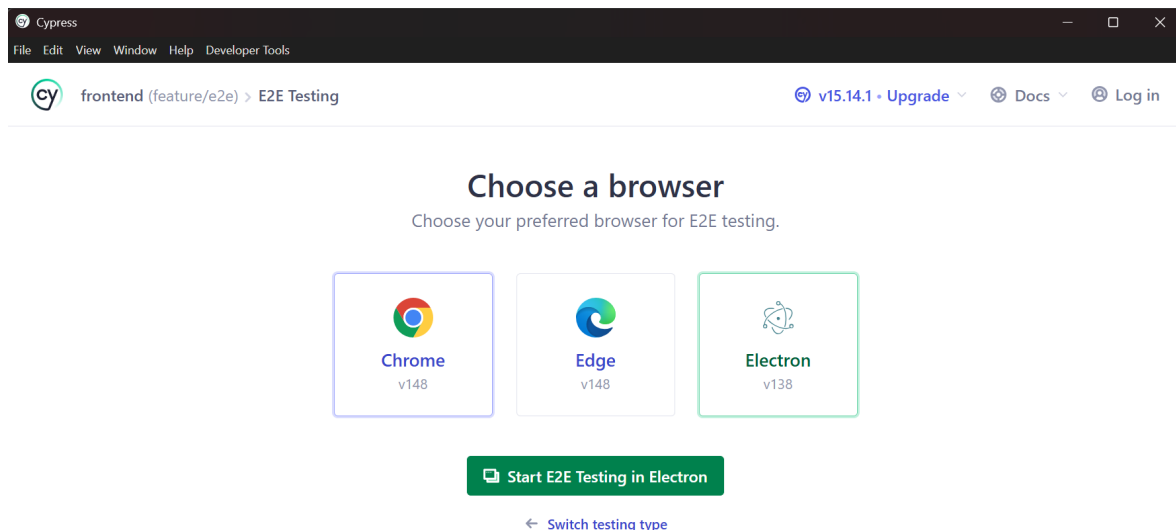
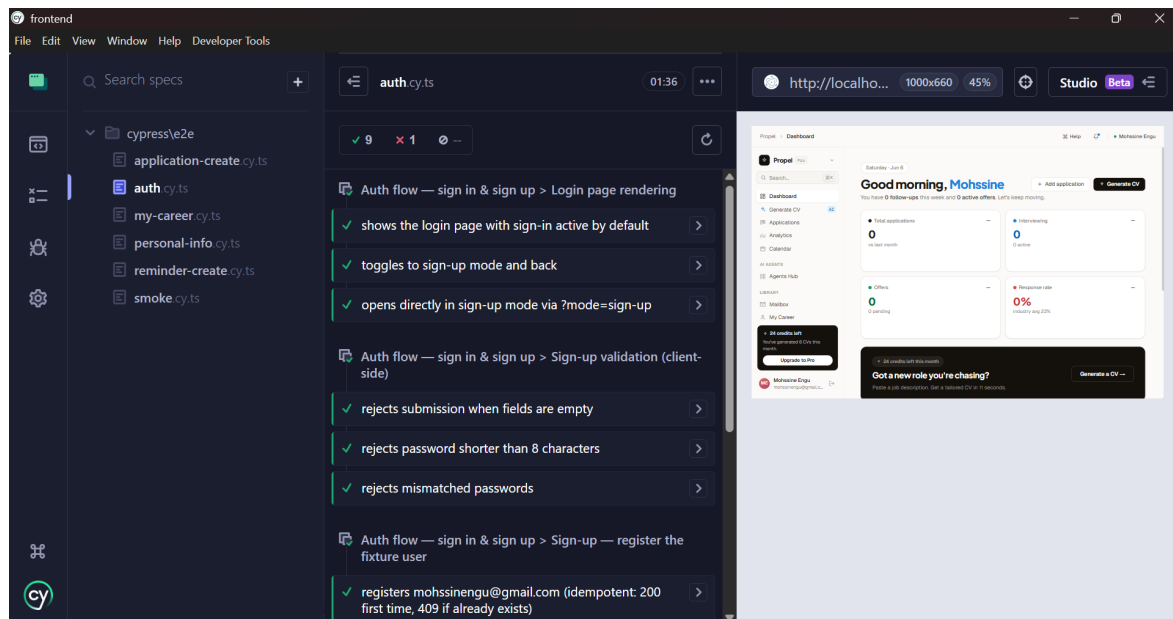


FIGURE 4.14 – Interface Cypress — Exécution de la suite E2E avec 24 tests passants

FIGURE 4.15 – Détail du fichier `auth.cy.ts` — Scénarios d'authentification

4.8.4 Tests de Performance — k6

Les tests de performance mesurent le comportement du système sous charge pour détecter les goulots d'étranglement avant la mise en production.

Infrastructure et configuration

L'outillage repose sur **k6** (Grafana) avec une architecture modulaire :

- **config/env.js** : Centralise l'URL de base (`http://localhost:8080` par défaut, configurable via `BASE_URL`), la carte des endpoints et les identifiants de l'utilisateur seedé (`testuser/test1234`).
- **lib/helpers.js** : Fournit `buildUser(runId, vu, iter)` pour générer des emails uniques par VU et itération (`perf_{timestamp}_v{vu}_i{iter}@cvgen-loadtest.local`), ainsi que les en-têtes JSON communs.
- **lib/auth.js** : Fournit `loginAs(email, password)` qui effectue `POST /api/auth/login` et maintient le cookie de session par VU pour les scénarios authentifiés.

L'infrastructure de test utilise une **base de données de test dédiée** et un **realm Keycloak isolé** pour ne pas polluer les données de développement.

Scénarios de charge

Six scénarios couvrent les chemins critiques de l'application :

TABLE 4.5 – Scénarios k6 et seuils de performance

Scénario	VUs	Iter.	Seuils (p95)
register.js	10	10	p95 < 10000ms, taux succès > 90%
login.js	10	50	p95 < 5000ms, taux succès > 90%
me.js	10	20	p95 < 500ms, taux succès > 95%
applications-list.js	10	10	p95 < 1500ms, taux succès > 95%
cvprofiles-create.js	10	5	p95 < 2000ms, taux succès > 90%
journey.js	10	1	taux succès > 90% (par étape)

Les scénarios ont été conçus pour tester des aspects différents :

- **register.js** : Mesure la latence de création d'utilisateur dans Keycloak et la publication de l'événement Kafka vers le User Service.
- **login.js** : Mesure le débit du token endpoint Keycloak sous charge concurrente (50 connexions partagées entre 10 VUs).
- **me.js** : Mesure la performance du chemin de validation de cookie sans appel en amont (seuil le plus strict : p95 < 500ms).
- **applications-list.js** : Mesure la planification de requêtes SQL complexes avec combinaisons aléatoires de filtres (`statuses`, `search`, `appliedFrom`, `appliedTo`) — permet de détecter les index manquants.
- **cvprofiles-create.js** : Mesure la latence d'écriture PostgreSQL combinée à la publication Kafka — permet d'isoler le goulot d'étranglement par comparaison avec `register.js`.
- **journey.js** : Exécute le parcours utilisateur complet (inscription → connexion → validation de session → création de profil CV → liste de candidatures) en parallèle sur 10 VUs. Ce scénario révèle les problèmes d'intégration invisibles aux tests d'endpoint unique : lag du consommateur Kafka, problèmes de cookie `SameSite`, et désynchronisation d'état entre services.

Chaque scénario définit des **métriques personnalisées k6** (`Rate` et `Counter`) pour le taux de succès et les erreurs par catégorie (4xx, 5xx, conflits), permettant une analyse fine des résultats.

```

PS C:\Users\hp\Ghailani-project\CV-Generator> k6 run tests/performance/scenarios/register.js

[WARN][0001] Thresholds on metrics http_req_duration, http_req_failed, register_success_rate have been crossed

Grafana

execution: local
script: tests/performance/scenarios/register.js
output: -

scenarios: (100.00%) 1 scenario, 10 max VUs, 10m30s max duration (incl. graceful stop):
  * default: 10 iterations shared among 10 VUs (maxDuration: 10m0s, gracefulStop: 30s)

INFO[0000] [register-load] run id = mqle276j → target = http://localhost:8080/api/auth/register source=console
INFO[0015] [register-load] finished run id = mqle276j source=console

THRESHOLDS
http_req_duration
X 'p(95)<2000' p(95)=14.72s

http_req_failed
✓ 'rate<0.10' rate=0.00%

register_success_rate
✓ 'rate>0.90' rate=100.00%

TOTAL RESULTS
checks_total.....: 30      1.989653/s
checks_succeeded...: 100.00% 30 out of 30
checks_failed.....: 0.00%  0 out of 30

✓ status is 200
✓ response has success=true
✓ message confirms account created

CUSTOM
register_success_rate.....: 100.00% 10 out of 10

HTTP
http_req_duration.....: avg=14.37s min=14.18s med=14.37s max=14.96s p(90)=14.48s p(95)=14.72s
  { expected_response:true }...: avg=14.37s min=14.18s med=14.37s max=14.96s p(90)=14.48s p(95)=14.72s
http_req_failed.....: 0.00%  0 out of 10
http_reqs.....: 10      0.663218/s

EXECUTION
iteration_duration.....: avg=14.45s min=14.28s med=14.44s max=15.04s p(90)=14.56s p(95)=14.8s
iterations.....: 10      0.663218/s
vus.....: 1      min=1      max=10
vus_max.....: 10      min=10      max=10

NETWORK
data_received.....: 2.4 kB 157 B/s
data_sent.....: 2.9 kB 190 B/s

```

FIGURE 4.16 – Sortie k6 — Scénario `register.js` : création d'utilisateurs sous charge (10 VUs)

```

PS C:\Users\hp\Ghailani-project\CV-Generator> k6 run tests/performance/scenarios/login.js

      Grafana
    /  |  \
   /___\  \

execution: local
script: tests/performance/scenarios/login.js
output: -

scenarios: (100.00%) 1 scenario, 10 max VUs, 10m30s max duration (incl. graceful stop):
  * default: 50 iterations shared among 10 VUs (maxDuration: 10m09s, gracefulStop: 30s)

INFO[0000] [login-load] target = http://localhost:8080/api/auth/login as test@example.com source=console

■ THRESHOLDS

http_req_duration
✓ 'p(95)<5000' p(95)=3.77s

http_req_failed
✓ 'rate<0.10' rate=0.00%

login_success_rate
✓ 'rate>0.90' rate=100.00%

■ TOTAL RESULTS

checks_total.....: 150      12.68327/s
checks_succeeded...: 100.00% 150 out of 150
checks_failed.....: 0.00%   0 out of 150

✓ status is 200
✓ success flag is true
✓ access token returned

CUSTOM
login_success_rate.....: 100.00% 50 out of 50

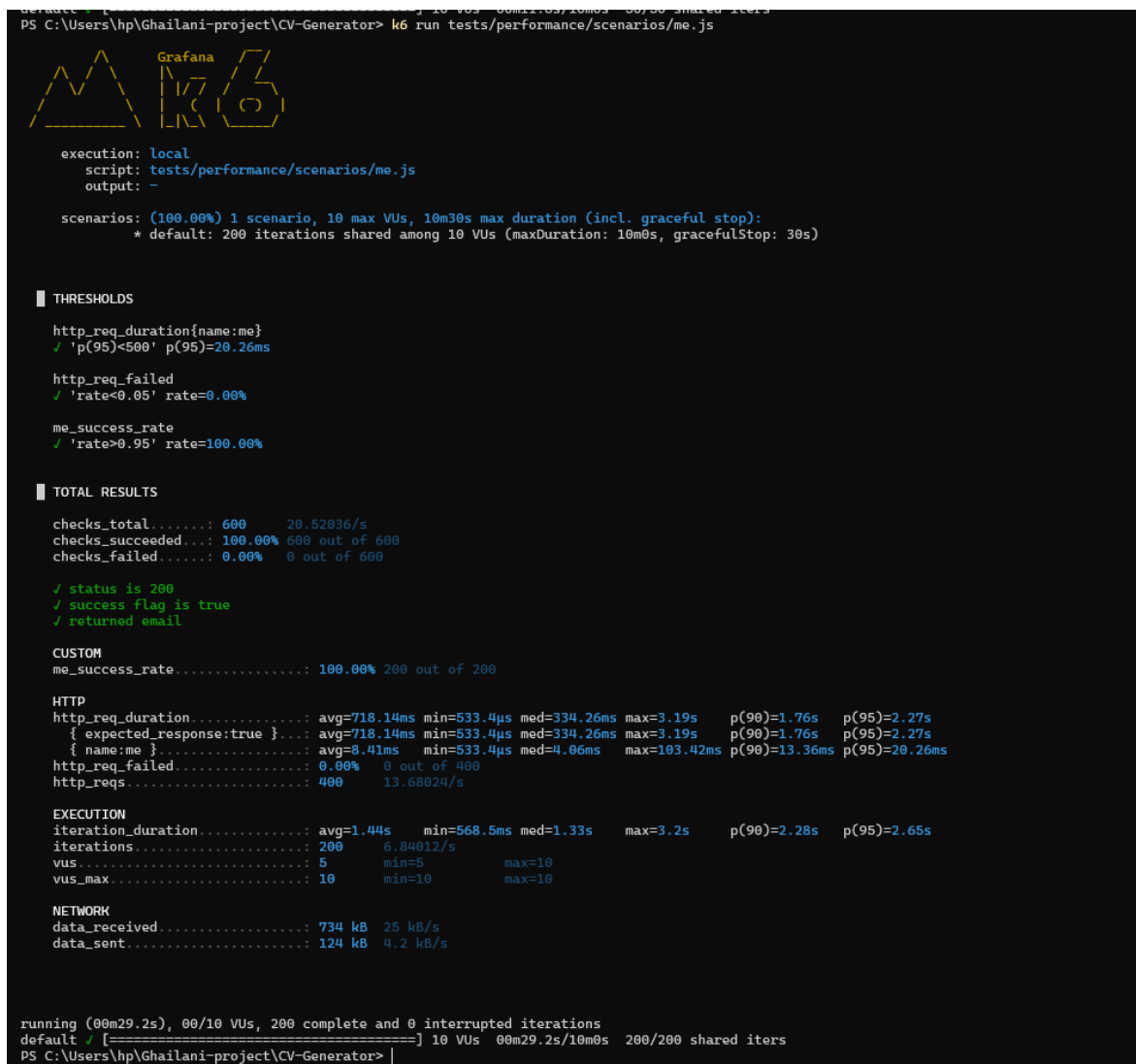
HTTP
http_req_duration.....: avg=2.23s min=645.13ms med=1.77s max=4.12s p(90)=3.56s p(95)=3.77s
{ expected_response:true }...: avg=2.23s min=645.13ms med=1.77s max=4.12s p(90)=3.56s p(95)=3.77s
http_req_failed.....: 0.00%   0 out of 50
http_reqs.....: 50      4.227757/s

EXECUTION
iteration_duration.....: avg=2.27s min=648.16ms med=1.77s max=4.2s p(90)=3.56s p(95)=3.78s
iterations.....: 50      4.227757/s
vus.....: 9      min=9      max=10
vus_max.....: 10      min=10      max=10

NETWORK
data_received.....: 104 kB  8.8 kB/s
data_sent.....: 11 kB  901 B/s

```

FIGURE 4.17 – Sortie k6 — Scénario login.js avec métriques et seuils



```

PS C:\Users\hp\Ghailani-project\CV-Generator> k6 run tests/performance/scenarios/me.js

      Grafana
    /  |  \
   /___|__\

execution: local
script: tests/performance/scenarios/me.js
output: -

scenarios: (100.00%) 1 scenario, 10 max VUs, 10m30s max duration (incl. graceful stop):
  * default: 200 iterations shared among 10 VUs (maxDuration: 10m0s, gracefulStop: 30s)

THRESHOLDS
http_req_duration{name:me}
✓ 'p(95)<500' p(95)=20.26ms

http_req_failed
✓ 'rate<0.05' rate=0.00%

me_success_rate
✓ 'rate>0.95' rate=100.00%

TOTAL RESULTS
checks_total.....: 600      20.52036/s
checks_succeeded...: 100.00% 600 out of 600
checks_failed.....: 0.00%   0 out of 600

✓ status is 200
✓ success flag is true
✓ returned email

CUSTOM
me_success_rate.....: 100.00% 200 out of 200

HTTP
http_req_duration.....: avg=718.14ms min=533.4µs med=334.26ms max=3.19s   p(90)=1.76s p(95)=2.27s
{ expected_response:true }...: avg=718.14ms min=533.4µs med=334.26ms max=3.19s   p(90)=1.76s p(95)=2.27s
{ name:me }.....: avg=8.41ms min=533.4µs med=4.06ms max=103.42ms p(90)=13.36ms p(95)=20.26ms
http_req_failed.....: 0.00%   0 out of 400
http_reqs.....: 400      13.68024/s

EXECUTION
iteration_duration.....: avg=1.44s min=568.5ms med=1.33s max=3.2s   p(90)=2.28s p(95)=2.65s
iterations.....: 200      6.84012/s
vus.....: 5 min=5 max=10
vus_max.....: 10 min=10 max=10

NETWORK
data_received.....: 734 kB 25 kB/s
data_sent.....: 124 kB 4.2 kB/s

running (00m29.2s), 00/10 VUs, 200 complete and 0 interrupted iterations
default ✓ [=====] 10 VUs 00m29.2s/10m0s 200/200 shared iters
PS C:\Users\hp\Ghailani-project\CV-Generator>

```

FIGURE 4.18 – Sortie k6 — Scénario `me.js` : validation du cookie de session ($p95 < 500\text{ms}$)

4.9 Conclusion

Ce chapitre a présenté en détail la réalisation technique du projet **Propel**. Nous avons couvert l'architecture physique et logicielle, les outils utilisés, le cycle DevSecOps, la gestion du code source, l'implémentation des différentes couches et le déploiement. Le projet démontre une maîtrise complète de la chaîne DevOps, de l'écriture du code jusqu'au déploiement automatisé en passant par la sécurité et la qualité.

Chapitre 5

Conclusion générale et perspectives

5.1 Conclusion générale

Le projet **Propel** nous a permis de concevoir et réaliser une plateforme complète de génération de CV assistée par intelligence artificielle. Ce projet d'intégration a mobilisé l'ensemble des compétences acquises durant notre formation, du développement web à l'architecture microservices, en passant par la conteneurisation, l'intégration continue, la sécurité applicative et l'observabilité.

L'approche DevOps adoptée, avec une organisation en microservices et un pipeline CI/CD complet intégrant qualité et sécurité, a permis de livrer une application robuste et industrialisée. La pipeline d'agents IA basée sur LangChain et LangGraph démontre la capacité à intégrer des technologies avancées d'intelligence artificielle dans une application concrète.

Sur le plan méthodologique, l'adoption de SCRUM avec des réunions quotidiennes et une communication continue via Google Meets et WhatsApp a favorisé une collaboration efficace au sein de l'équipe de six développeurs fullstack, chacun apportant son expertise dans son domaine de responsabilité secondaire.

5.2 Perspectives

Plusieurs perspectives d'amélioration et d'évolution sont envisageables pour **Propel** :

- **Déploiement cloud** : Migrer l'infrastructure vers un fournisseur cloud (AWS, Azure, GCP) avec orchestration Kubernetes pour une meilleure scalabilité.
- **Modèles de langage additionnels** : Intégrer d'autres fournisseurs de LLM (OpenAI, Anthropic, modèles open source en local) pour diversifier les capacités de génération.
- **Internationalisation** : Ajouter le support multilingue pour l'interface et la

génération de CV dans différentes langues.

- **Analyse avancée :** Développer des fonctionnalités d'analyse prédictive pour suggérer les offres d'emploi les plus pertinentes en fonction du profil de l'utilisateur.
- **Mode hors ligne :** Proposer une version allégée de l'application fonctionnant sans connexion Internet pour la consultation des CV et candidatures.
- **Place de marché :** Créer une plateforme de mise en relation entre candidats et recruteurs utilisant les données de profiling générées par l'application.

Bibliographie

Angular documentation, 2025. URL <https://angular.dev>.

Apache kafka documentation, 2025. URL <https://kafka.apache.org/documentation>.

Asp.net core documentation, 2025. URL <https://learn.microsoft.com/en-us/aspnet/core>.

Cypress documentation, 2025. URL <https://docs.cypress.io>.

Docker documentation, 2025. URL <https://docs.docker.com>.

Fastapi documentation, 2025. URL <https://fastapi.tiangolo.com>.

Grafana documentation, 2025. URL <https://grafana.com/docs>.

k6 documentation, 2025. URL <https://grafana.com/docs/k6>.

Keycloak documentation, 2025. URL <https://www.keycloak.org/documentation>.

Langchain documentation, 2025. URL <https://python.langchain.com>.

Postgresql documentation, 2025. URL <https://www.postgresql.org/docs>.

Prometheus documentation, 2025. URL <https://prometheus.io/docs>.

Sonarcloud documentation, 2025. URL <https://docs.sonarcloud.io>.

Terraform documentation, 2025. URL <https://developer.hashicorp.com/terraform/docs>.

Trivy documentation, 2025. URL <https://trivy.dev>.